

10. 开源云计算平台OpenStack



冯奕

2026



南京大學
NANJING UNIVERSITY

回顾(1)

需求和设计

- 云计算概念——定义；计算模式的发展；云计算模式；发展与推动力；特征、部署模型、服务模型；相关技术；三元认知论
- 云计算架构——极至的SOA架构；概念架构（二维视角，关注客户需求）、逻辑架构（关注如何实现：云体+云栈；技术体系结构）、物理架构（云数据中心+云操作系统）
- 云计算实例——华为云及实践——【实践1：OBS, ModelArts】

基础技术

- 云计算基础——分布式计算、分布式存储——【实践2：Flink, Ceph】
- 云计算基础——虚拟化技术、容器技术——【实践3：Docker】
- 云计算基础——计算机网络发展历程、云数据中心网络

实现

- 云数据中心——设计、特征，绿色节能、自动、高可用
- 云管理平台OpenStack 

- 云计算运维和云计算安全



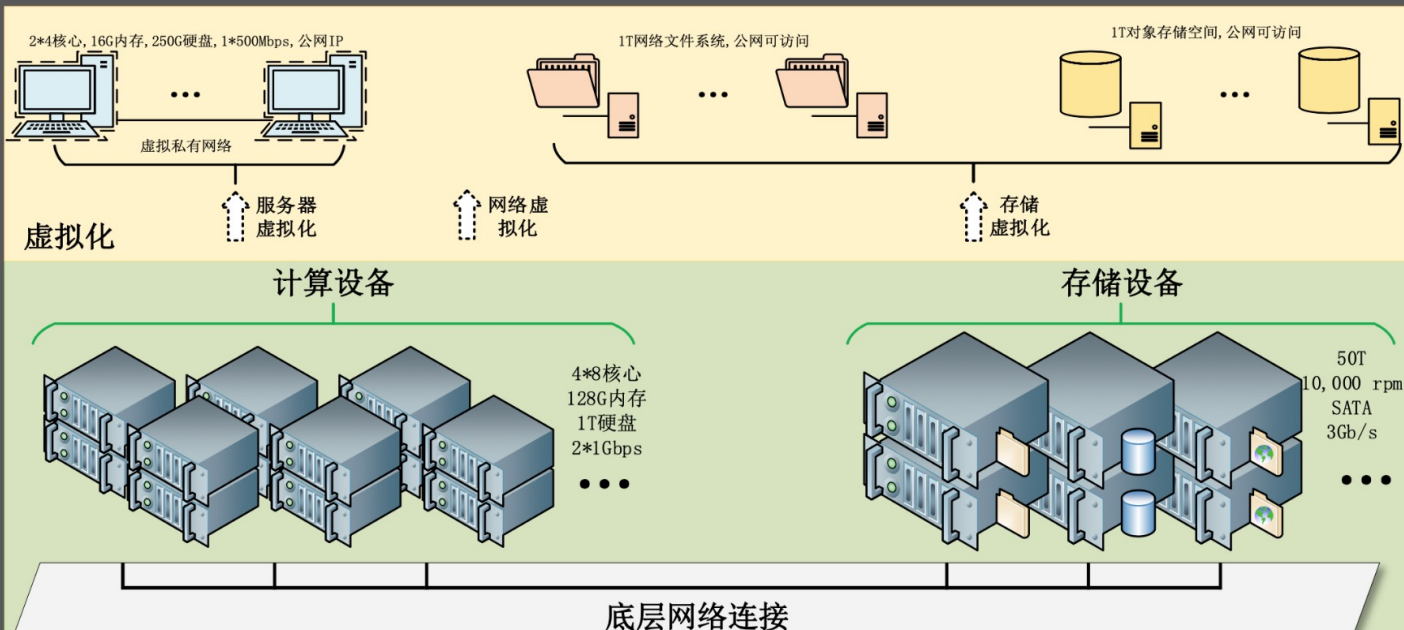
回顾(2)

3. 云操作系统

云平台操作系统

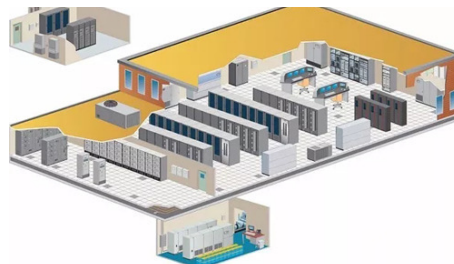
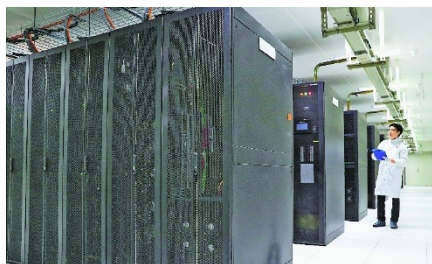
通过互联网提供友好的用户界面向用户展示并方便用户使用各类服务, 接收用户请求、响应用户请求等

物理资源、虚拟资源、用户、服务、数据、安全等
监控、管理、计费等



执行面

2. 云数据中心



Openstack的定位:
云计算系统的**控制面**

- 自身不提供虚拟化技术
- 调用多种技术实现多资源池管理
- 对外提供统一管理接口



目录

- OpenStack
 - 第一印象
 - 加深印象
 - 从概念模型到物理部署
 - 案例：虚拟机创建过程
- 云控制器：Keystone、Nova及其他服务
- 计算节点：Hypervisor、Magnum
- 存储节点：Swift, Cinder
- 网络节点：Neutron
- 重审案例
- 部署
 - DevOps
 - OpenStack和DevOps



第一印象—OpenStack是什么？

OpenStack引领新一代数据中心：为下一代数据中心操作系统赋能

需求

专注于快速、灵活和自动化地提供服务



设计

单一、大型、灵活、可扩展和自动化的单元

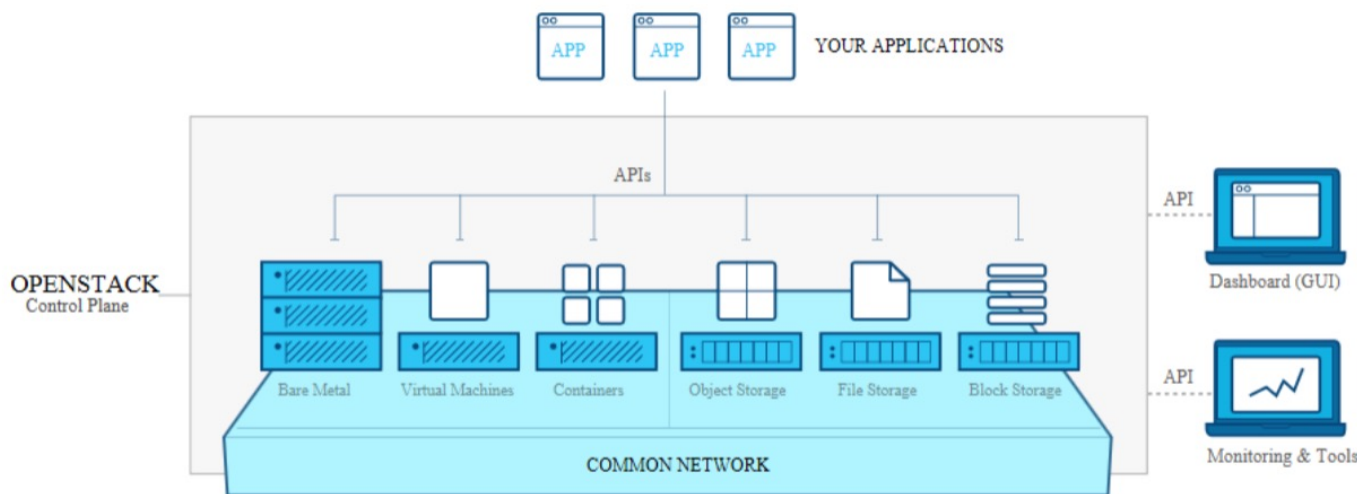


实现

可编程、可扩展、多租户感知的基础架构

- 是开源云计算平台，可控制整个数据中心的大型计算、存储和网络资源池。
- 管理员能够通过Web界面、命令行或API接口控制、配置资源；用户可以通过Web界面使用资源。

What is OpenStack?



OpenStack is a cloud operating system that controls large pools of compute, storage, and networking resources throughout a datacenter, all managed through a dashboard that gives administrators control while empowering their users to provision resources through a web interface.



加深印象 (1)

- 既是一个**开源**软件，也是一个开源社区，里面包含很多开源子项目
- 提供了一个部署云的操作平台和工具集 文档: <https://docs.openstack.org/install-guide/index.html>
- 用OpenStack易于构建虚拟计算或存储服务云，可为公有、私有云（无论大小）提供**可扩展**、**灵活**的云计算



计算资源管理

OpenStack可以规划并管理大量虚拟机、容器，从而允许企业或服务提供商按需提供计算资源



存储资源管理

OpenStack可以为云服务或云应用提供所需的文件存储、对象存储、块存储的资源



网络资源管理

IP地址的数量、路由配置、安全规则将爆炸式增长；传统的网络管理技术无法真正高扩展、高自动化地管理下一代网络

IaaS



PaaS

开放

Open

理念：不重复造轮子，站在巨人的肩膀上

措施：开源，尽最大可能重用开源项目

设计理念



参与openstack项目的主要企业: <https://www.openstack.org/community/supporting-organizations/>

加深印象 (1)

鲲鹏服务器
昇腾芯片
Euler操作系统



加深印象 (1)

- 既是一个**开源**软件，也是一个开源社区，里面包含很多开源子项目
- 提供了一个部署云的操作平台和工具集 文档: <https://docs.openstack.org/install-guide/index.html>
- 用OpenStack易于构建虚拟计算或存储服务云，可为公有、私有云（无论大小）提供**可扩展**、**灵活**的云计算



计算资源管理

OpenStack可以规划并管理大量虚拟机、容器，从而允许企业或服务提供商按需提供计算资源



存储资源管理

OpenStack可以为云服务或云应用提供所需的文件存储、对象存储、块存储的资源



网络资源管理

IP地址的数量、路由配置、安全规则将爆炸式增长；传统的网络管理技术无法真正高扩展、高自动化地管理下一代网络



设计理念

开放

Open

理念：不重复造轮子，站在巨人的肩膀上

措施：开源，尽最大可能重用开源项目

灵活

Flexible

大量使用插件化方式进行架构设计与实现

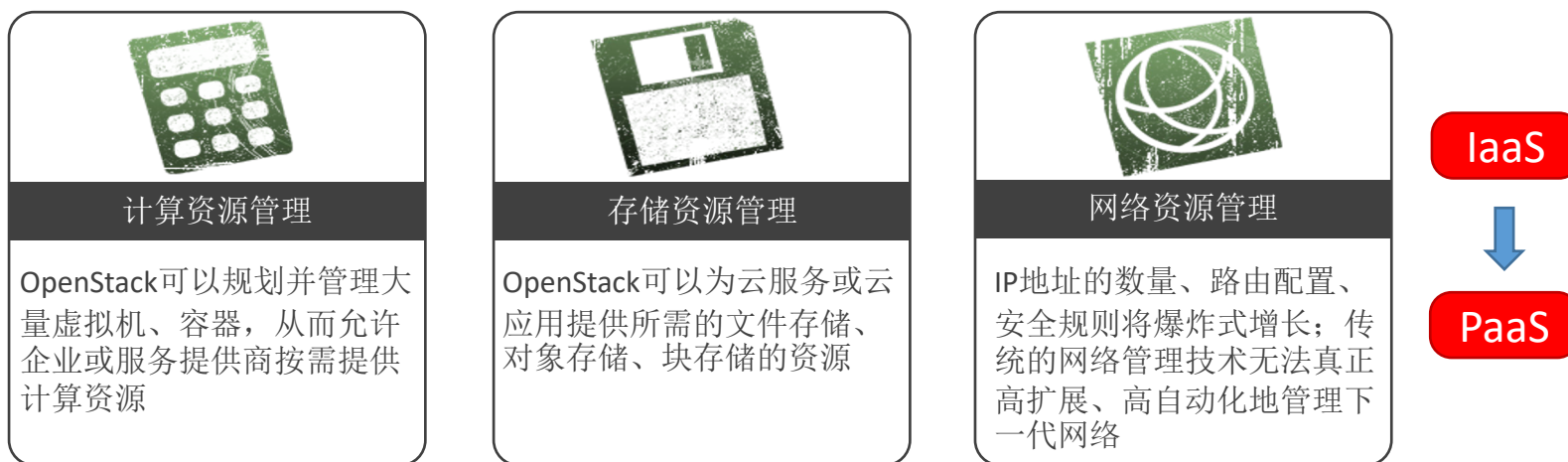
不使用任何不可替代的私有/商业组件



参与openstack项目的主要企业: <https://www.openstack.org/community/supporting-organizations/>

加深印象 (1)

- 既是一个**开源**软件，也是一个开源社区，里面包含很多开源子项目
- 提供了一个部署云的操作平台和工具集 文档: <https://docs.openstack.org/install-guide/index.html>
- 用OpenStack易于构建虚拟计算或存储服务云，可为公有、私有云（无论大小）提供**可扩展**、**灵活**的云计算



设计理念

开放

Open

理念：不重复造轮子，站在巨人的肩膀上

措施：开源，尽最大可能重用开源项目

灵活

Flexible

大量使用插件化方式进行架构设计与实现

不使用任何不可替代的私有/商业组件

可扩展

Scalable

由多个相互独立的项目组成

每个项目包含多个独立组件

无中心架构，无状态架构



参与openstack项目的主要企业: <https://www.openstack.org/community/supporting-organizations/>

加深印象（2）—深度类比“计算机操作系统”

- 操作系统是一组**主管并控制**计算机操作、运用和运行硬件、软件资源和**提供公共服务**来组织用户交互的相互关联的系统软件程序。
- 云平台综合管理系统指构架于服务器、存储、网络等**基础硬件资源**和单机操作系统、中间件、数据库等**基础软件**之上，**管理**海量的基础硬件、软件资源并**提供底层公共服务**的软件程序。

传统操作系统的功能

1. 资源抽象：单台物理设备的计算、存储、网络、I/O等
2. 资源分配与负载调度：各种CPU调度算法、内存调度算法
3. 应用生命周期管理：启动后的不同状态、关闭后的管理
4. 系统运维：资源整理
5. 人机交互：用户

如OpenStack的云操作系统的功能

1. 资源抽象：云数据中心的大量计算、存储等设备
2. 资源分配与负载调度：以虚拟机、容器等单位
3. 应用生命周期管理：虚拟机、云应用生命周期
4. 系统运维：更高层面的系统运维
5. 人机交互：管理员、应用开发者、服务终端用户



加深印象 (2) —深度类比 “计算机操作系统”

	管理的资源	运行的例程	接口功能	包含的分布式库
传统OS	一台或多台电脑	调度器、虚拟内存分配、文件系统和中断处理程序	提供管理底层硬件的库函数	标准分布式库和软件包
云OS	云数据中心资源	提供更多附加功能，虚拟机的分配和释放、任务的分配与融合	为云应用提供统一的标准资源管理API	为分布式应用提供自主扩展和灵活调度的软件支持

全面网络化

更多的安全考虑

计算可扩充性考虑

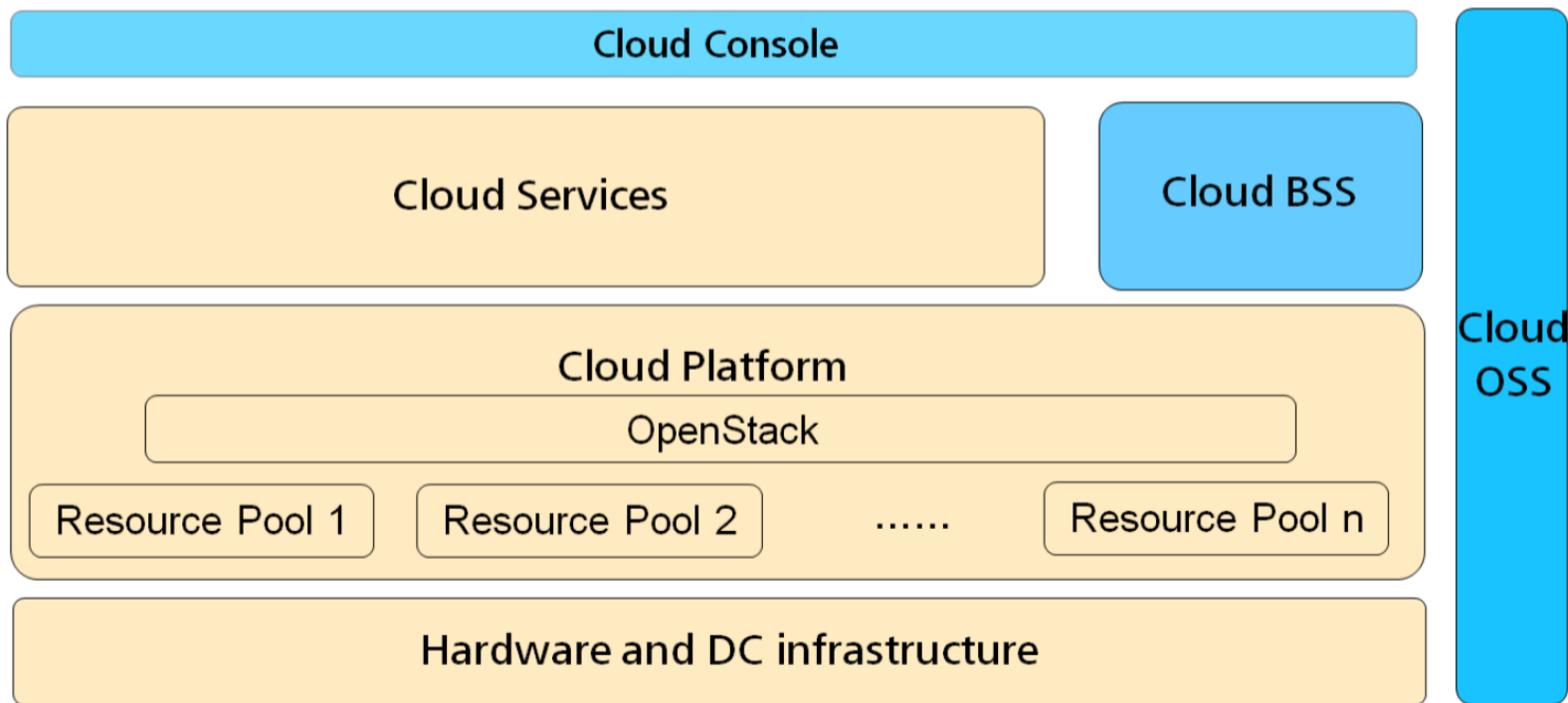


“云操作系统”的三种不同指代

1. 类虚拟化中为适配云计算需求而被修改的传统操作系统
2. 在每一台物理设备上或集群里管理虚拟机的监控器VMM
3. 云平台或数据中心操作系统

加深印象（3）——在云计算中地位

- 重要地位：赋能安全、灵活、易扩展的全面自动化数据中心
- 除云操作系统，构建云计算还需要其他东西：基础硬件+基础软件+上层应用程序 → 同等重要



Apache CloudStack™

Open Source Cloud Computing™



<https://cloudstack.apache.org/>

EUCALYPTUS

Elastic Utility Computing Architecture for
Linking Your Programs To Useful Systems

<https://www.eucalyptus.cloud/>



从概念模型到物理部署 (1)

对云OS的功能需求 → 概念模型设计 → 各个功能模块及其之间的交互

终端用户能够申请和使用云计算中的虚拟机、存储和网络资源，并计量用量

管理员能够监控云平台状态、计量用户使用资源情况、方便地扩展云规模等

Keystone

Horizon

Nova

Glance

Swift

Cinder

Neutron

Telemetry

Manila

Heat

功能类别	具体描述
身份认证	管理用户信息并提供认证服务
仪表盘	提供图形化用户界面，包括普通用户和管理员用户
计算	管理计算节点，并管理虚拟机创建、运行到销毁地整个生命周期
镜像	管理平台中所有的操作系统镜像，能够进行存储、查询、读取等操作
对象存储	整合一定规模的存储设备，构建对象存储服务，包括增删改查等操作
块存储	整合一定规模的存储设备，对外提供卷服务，包括新建卷、删除卷等
网络	能够动态为虚拟机配置公网IP地址；能在各种场景下为用户建立私有网络
计量	对每一个用户使用资源提供度量服务，包括指标管理和根据指标报警
文件共享	整合一定规模的存储设备对外提供横向可扩展的文件共享系统
编排	方便用户根据自己的需求创建虚拟资源并使用，只需要编写配置文件

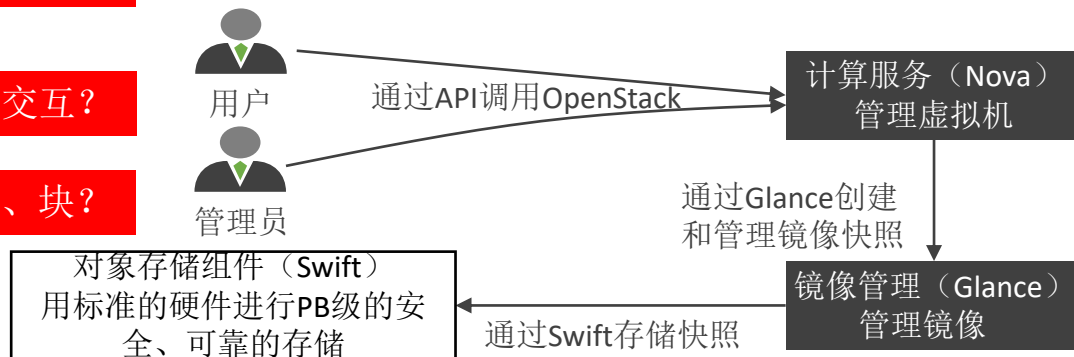
物理上

问题3：各组件如何关联？

业务上

问题2：各组件如何交互？

问题1：文件、对象、块？

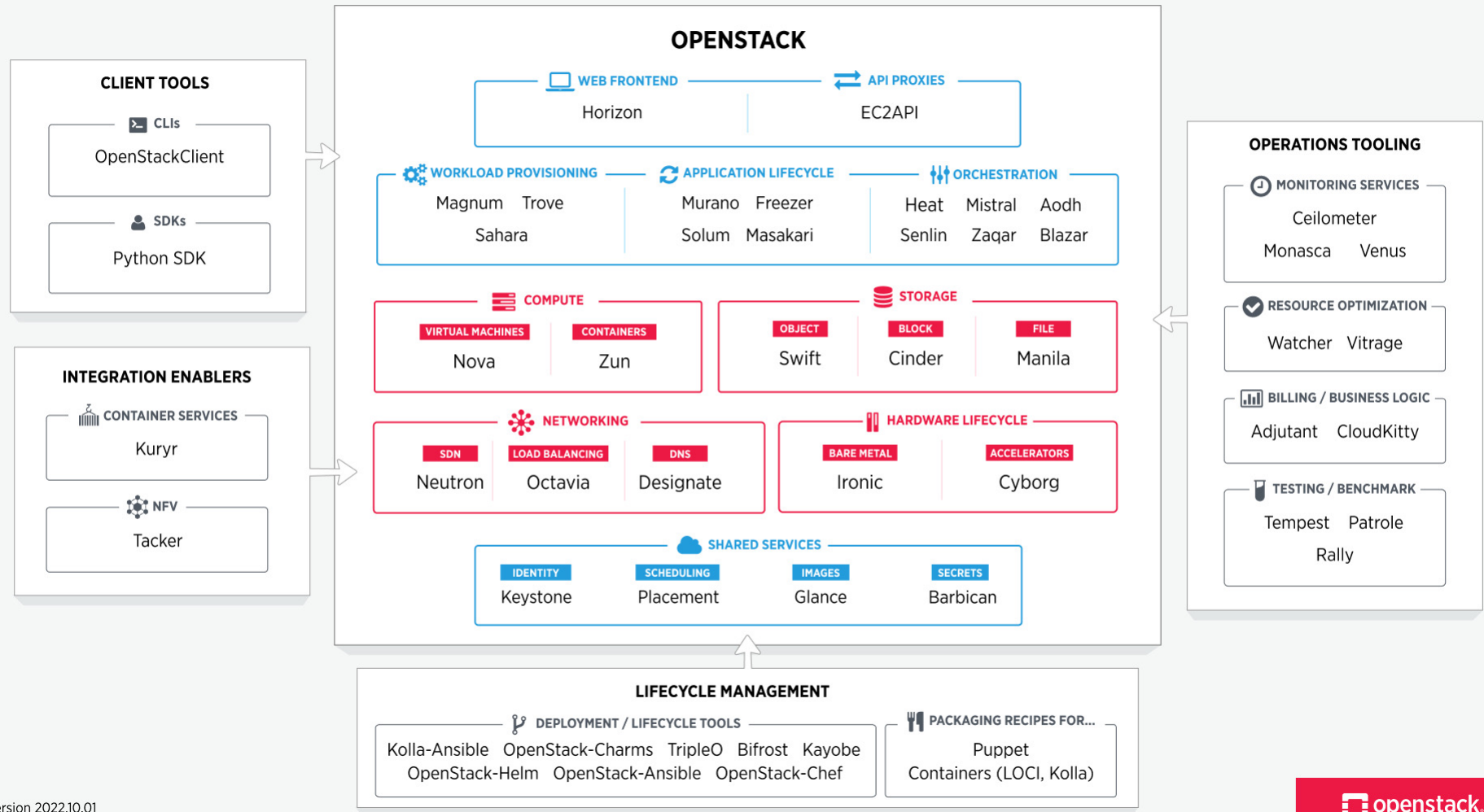


Map of OpenStack Projects

灵活 | Flexible

大量使用插件化方式
进行架构设计与实现

不使用任何不可替代
的私有/商业组件



从概念模型到物理部署 (2)

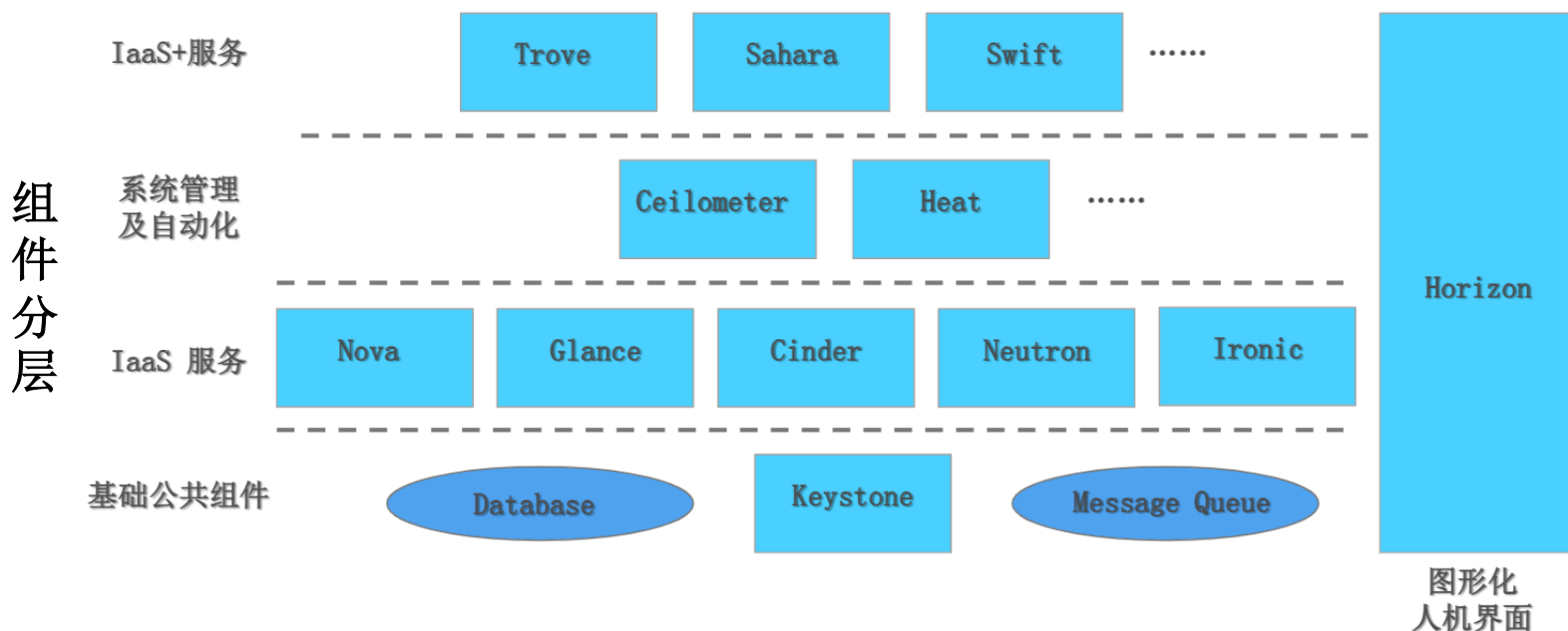
- 除了以上各功能性服务组件，还有两个非常核心的基础组件

？交互的物质基础——媒介

- 消息队列：中心化的消息交换器，用于在服务组件不同模块传递消息；队列的优势在于可以**缓冲请求**
- 数据库：构建时和运行时状态信息，例如可以使用的实例类型、正在使用的实例、可用网络等，也是各个组件间**共享信息**的重要组件

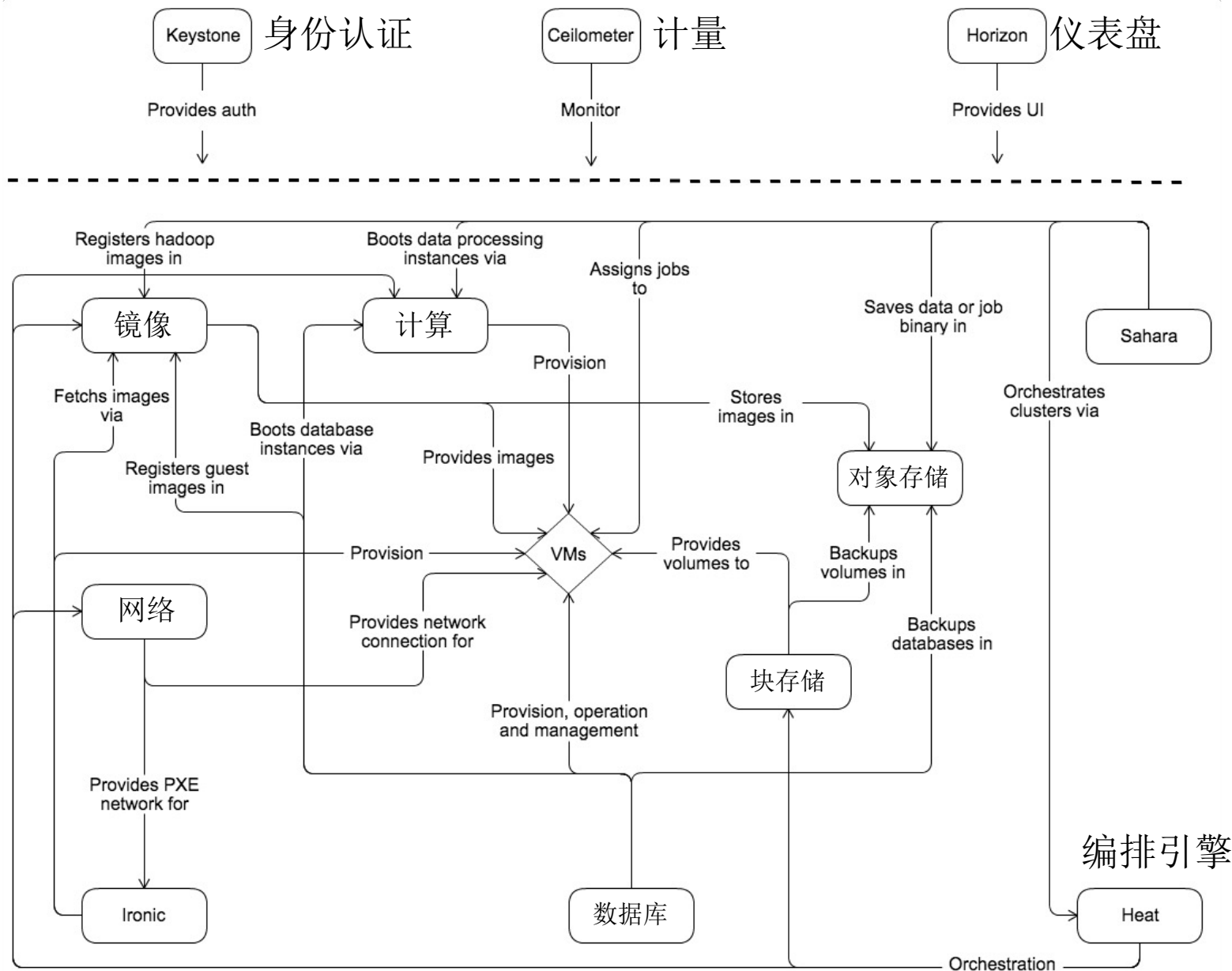
RESTful API

- 服务目录：所有服务向其注册，了解OpenStack的所有服务——Keystone承担注册中心角色



从概念模型到物理部署 (3)

各个组件之间详细的交互关系描述



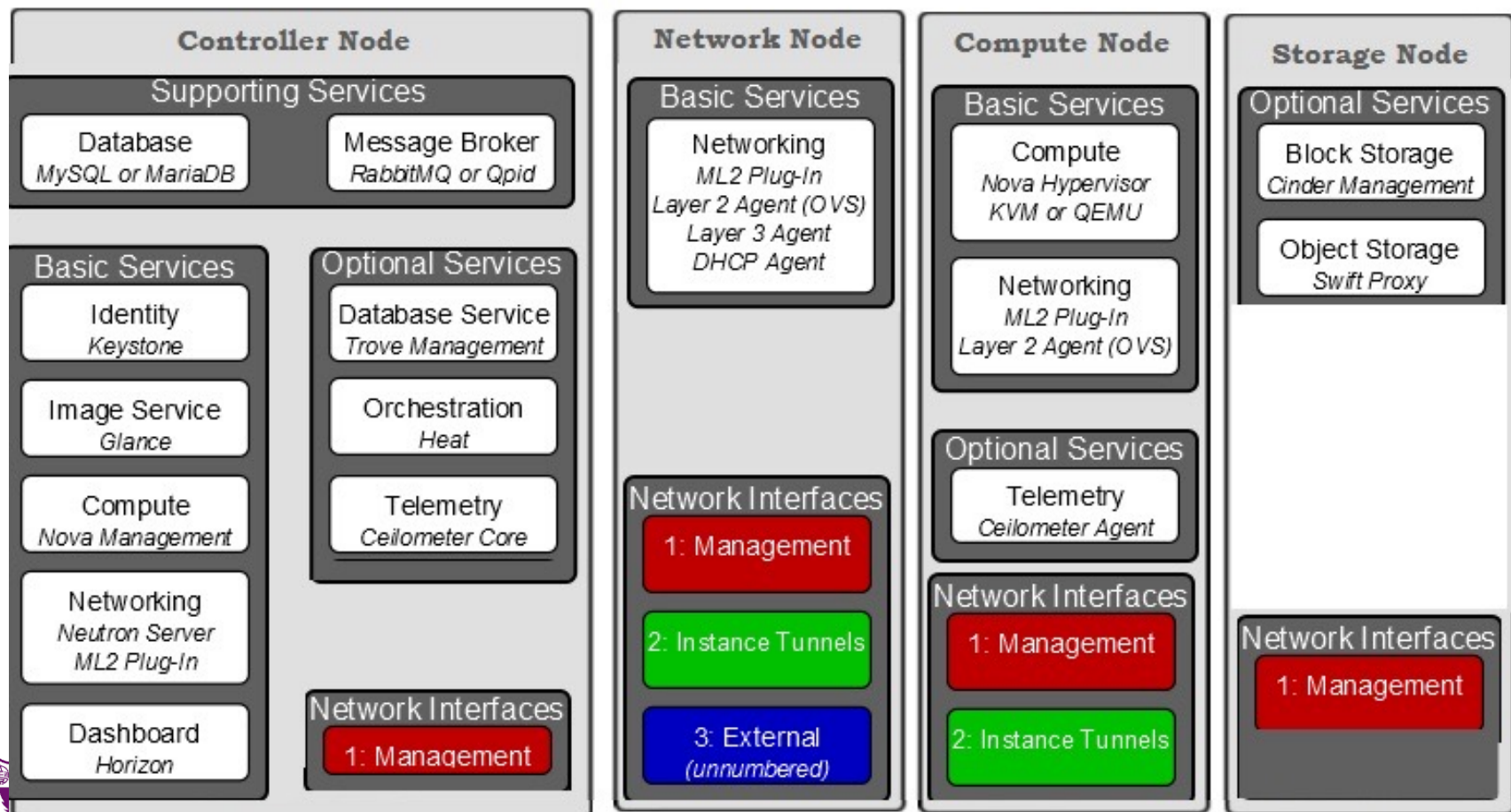
从概念模型到物理部署 (4)

单台计算机设备：CPU、内存、硬盘、总线？ $\Leftrightarrow$ 云数据中心

物理上如何关联？

所有组件分布在四种节点上：云控制器、网络节点、计算节点、存储节点；**网络连接**

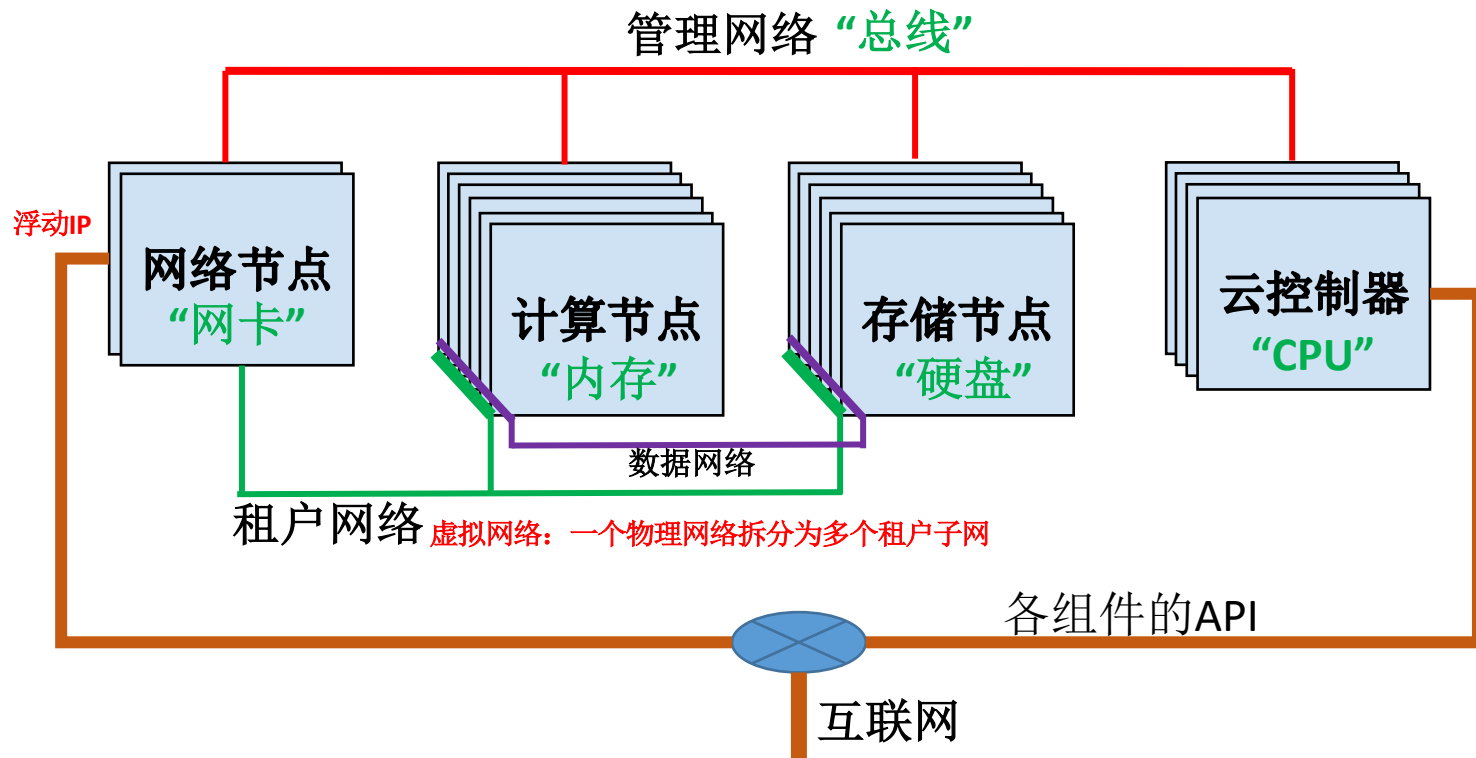
OpenStack 4-Node Architecture



控制节点中封装了很多Openstack的服务

从概念模型到物理部署 (5)

- 四种节点如何通过物理网络连接在一起？
 - 管理网络：Openstack服务之间的通信，包括消息队列、数据库和API
 - 租户数据网络：为租户创建的**虚拟网络**提供物理路径
 - 存储网络：虚拟机和存储节点之间的物理连接
 - 对外的网络：互联网连接+对外的API网络
- 使用隔离的**物理网络**处理不同类型的网络流量：同一个节点在不同网络中使用不同接口

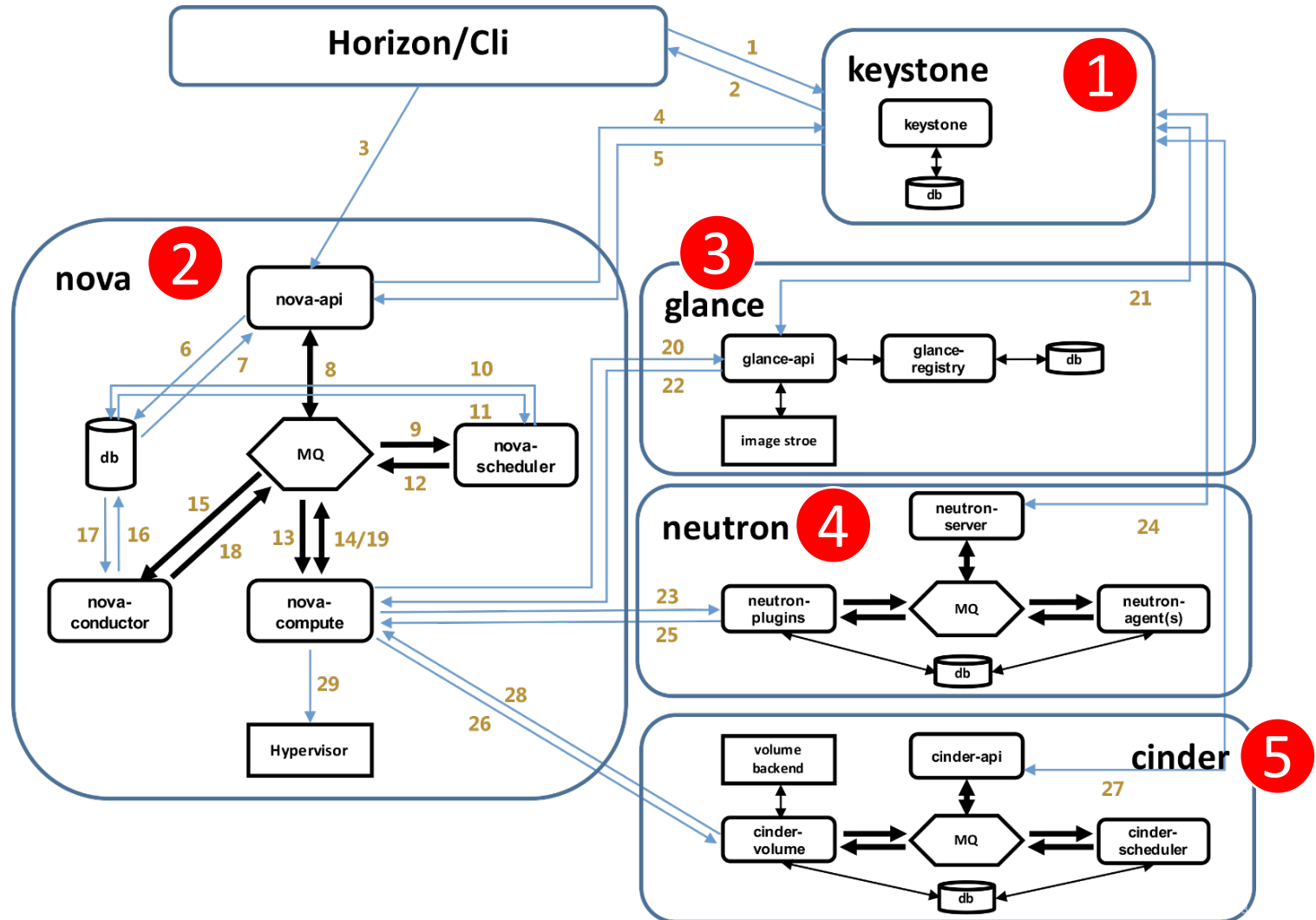


从概念模型到物理部署（6）

- 物理模型设计——估算硬件容量
 - 例如：在OpenStack环境中运行200个虚拟机
 - **CPU评估**
 - 逻辑CPU数：物理CPU*核数*超线程数；一个逻辑CPU对应一个虚拟CPU；一个虚拟CPU支持多个虚拟机【影响性能；可以使用频数计算】；一个虚拟机可以使用多个虚拟CPU，但不能多于逻辑CPU数；
 - 可以超额分配，但是不能超额执行；超分：单台物理机中虚拟机个数 * 频数 > 物理频数
 - **内存评估**
 - 单台物理机虚拟机个数*虚拟机最大动态分配内存+主机内存用量
 - **网络评估**
 - 网卡带宽 > 单个虚拟机带宽*虚拟机个数；公共IP地址个数；浮动IP地址个数
 - **存储评估**
 - 虚拟机的临时存储+永久存储；存储节点的对象存储、块存储和文件共享存储
- **最佳实践**
 - 分析需求、考虑需求增长情况；持续跟踪每一个服务单元、丢弃和替换服务单元等



案例：虚拟机创建过程



目录

- OpenStack简介
 - 第一印象
 - 加深印象
 - 从概念模型到物理部署
 - 案例：虚拟机创建过程
- 云控制器：Keystone、Nova及其他服务
- 计算节点：Hypervisor、Magnum
- 存储节点：Swift, Cinder
- 网络节点：Neutron
- 重审案例
- 部署
 - DevOps
 - OpenStack和DevOps



云控制器—总览

• 基础架构服务

- 不属于OpenStack对外提供的公共服务，但被多个Openstack组件使用
- 消息队列：必须是集群式的；RabbitMQ, ZeroMQ, Qpid
- 数据库：OpenStack的环境数据，使用数据等；MySQL和MongoDB

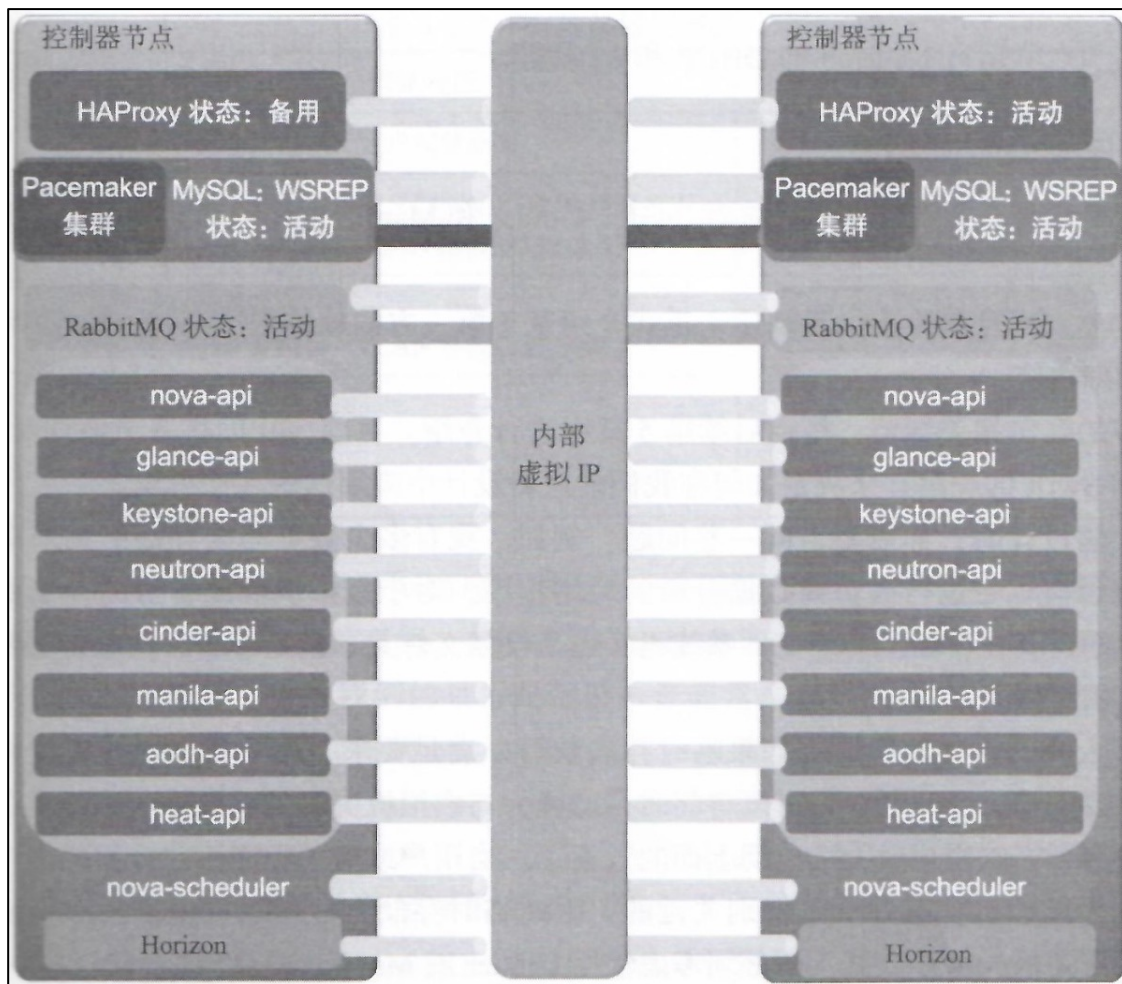
• OpenStack服务

- 身份认证
- 编排引擎
- 计量服务
- 仪表盘
-

各服务的Restful API

所有服务不一定要部署在同一台物理设备

运行每一类服务的都可以是一个独立的集群



云控制器——物理部署

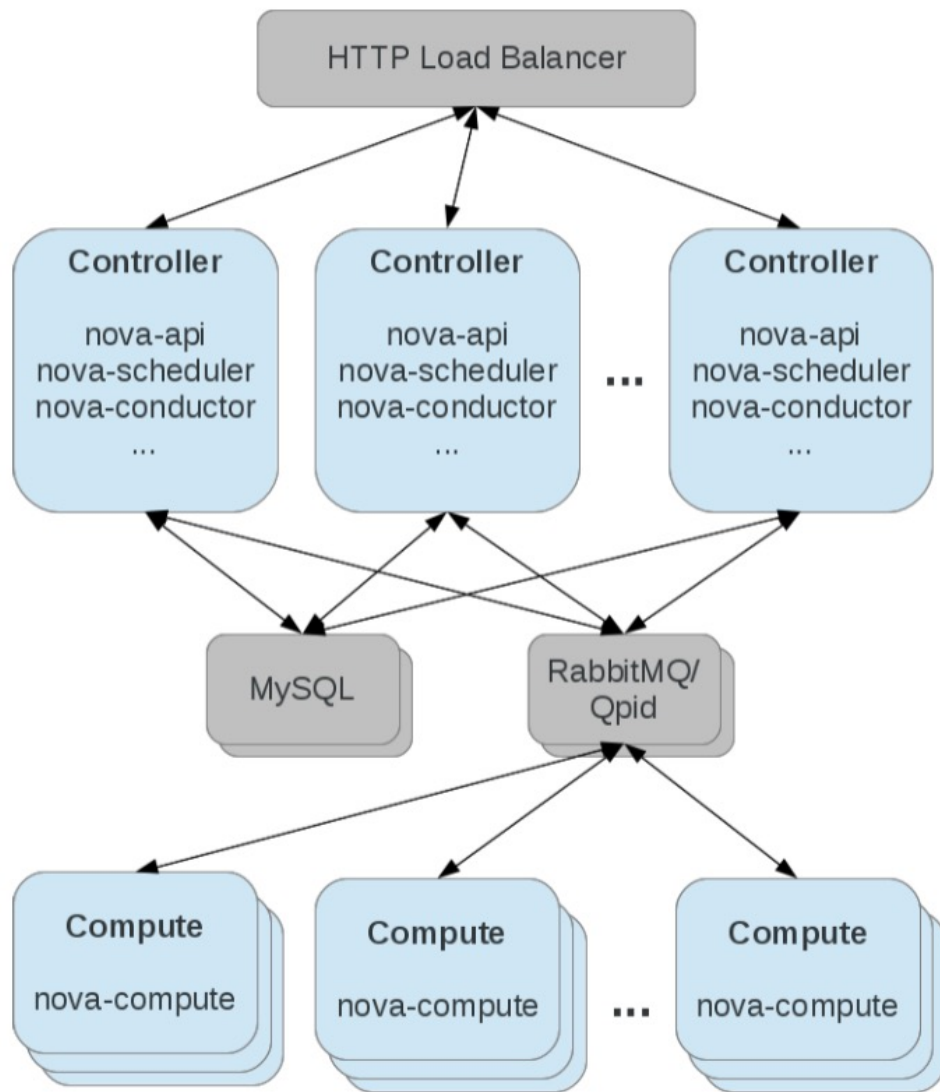
- Openstack采用 无中心结构、节点无本地状态
- 控制器(除基础架构服务)使用对称高性能集群
 - 请求聚合+负载均衡
 - 每一台物理设备运行相同的服务
- 控制器节点很容易水平扩展
- 数据库、消息队列可采用非对称高可用集群

★ **集群：** 两台或多台服务器的能力聚合就是服务器集群

非对称集群： 通常用于高可用目的，比如扩展数据库、消息系统或者文件系统的读写能力。备用服务器只有在主服务器发生故障的时候才会接管系统。备用服务器可以看作悄悄执行的观察者，集群需要配置故障切换机制。

如何保证所有节点状态一致？

对称集群： 所有节点都是活的，都参与请求的处理，当节点出现故障后，被集群丢弃，其他节点接管其工作负载并继续处理事务。典型的就负载均衡集群，所有节点共享工作负载，增强性能和服务的扩展性。



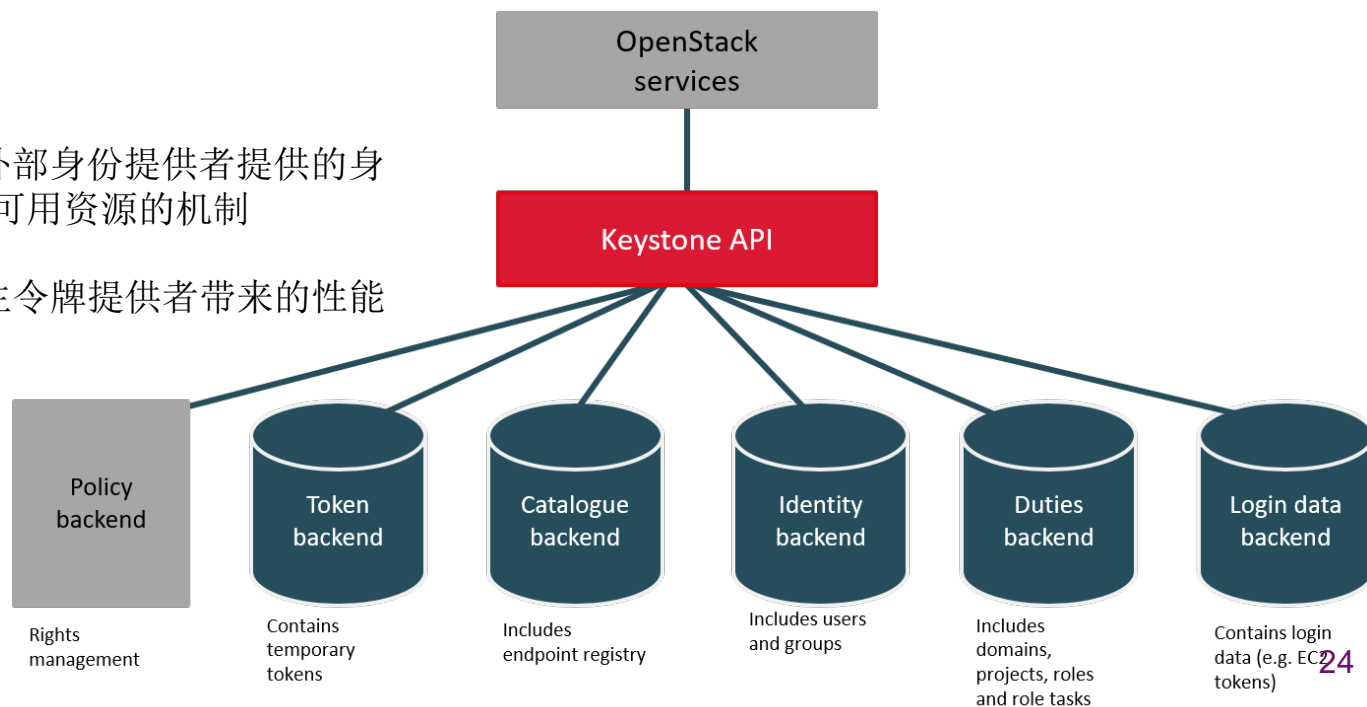
云控制器——Keystone服务

- 身份认证和服务目录（所有其他服务需向Keystone注册API端点），由多个Provider组成
 - 身份提供者Identity Provider：服务用户、管理员用户、终端用户
 - 资源提供者resource Provider：project、domain
 - 认证提供者Authorization Provider：用户和用户组和他们角色之间的关系
 - 令牌提供者Token Provider：访问openstack服务需要一个有效的令牌，通过身份提供者的认证后获得令牌
 - 目录提供者Catalog Provider：维护服务和API端点的目录
 - 策略提供者Policy Provider：策略由多条规则组成，每条规则定义了哪些用户和角色被允许访问哪些资源

高级特性

1. 联邦keystone：使用外部身份提供者提供的身份服务访问服务提供商可用资源的机制

2. Fernet令牌：解决原生令牌提供者带来的性能问题



云控制器——Nova管理组件及API

- Nova是Openstack中提供计算资源服务的项目，是OpenStack最核心的项目
- 负责：虚拟机和其他计算资源的生命周期管理
- 不负责：承载虚拟机的物理机自身的管理和全面的系统状态监控
- 运行在云控制器上的组件
 - Nova-api
 - 云控制器中的编排引擎；通过API将消息写入数据库和消息队列向其他守护进程传递消息
 - Nova-conductor
 - 代表计算节点上nova-compute执行数据库操作，提供数据库访问隔离；将来自计算节点请求并行化
 - Nova-scheduler
 - 专门的调度算法；确定虚拟机创建的最佳放置位置；支持过滤器检查计算节点上资源可用性，通过加权机制过滤出计算节点列表，再确定启动虚拟机的最佳位置；支持自定义度量标准和策略考量配置
 - 在生产环境下，配置单独的存储主机，其调度器运行在存储主机上
- Nova-compute则运行在计算节点上

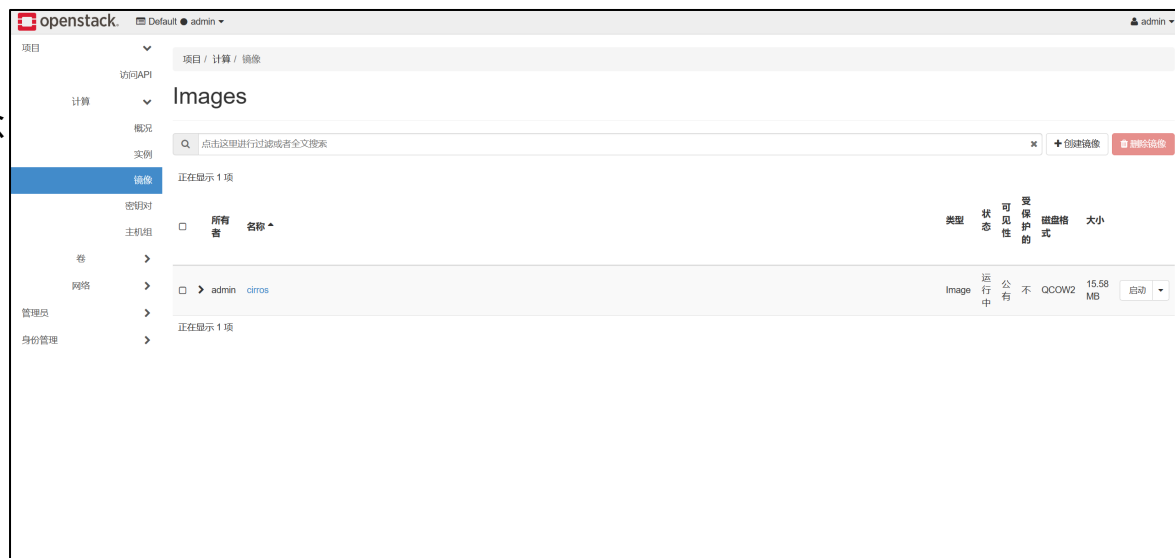


云控制器——其他服务

- 云控制器托管用于镜像管理的Glance服务
 - Glance API提供外部REST接口，用于查询虚拟机镜像及相关元数据
 - Glance registry将镜像元数据存储于数据库中，并利用存储后端实际存储镜像
 - 例如使用Swift对象存储作为镜像存储后端

- 网络服务采用类似Nova的部署概念

- 网络API驻留在云控制器中
 - 独立配置网络服务器节点



- 仪表盘服务

- 运行在Apache Web服务器后端
 - 可将其运行在可以访问API端点的单独节点上降低云控制器负载

- 计量服务

- 跟踪资源使用情况，计算资源利用率
 - 多种使用目的：计费、容量规划、按需求和吞吐量的虚拟基础架构自动扩展等



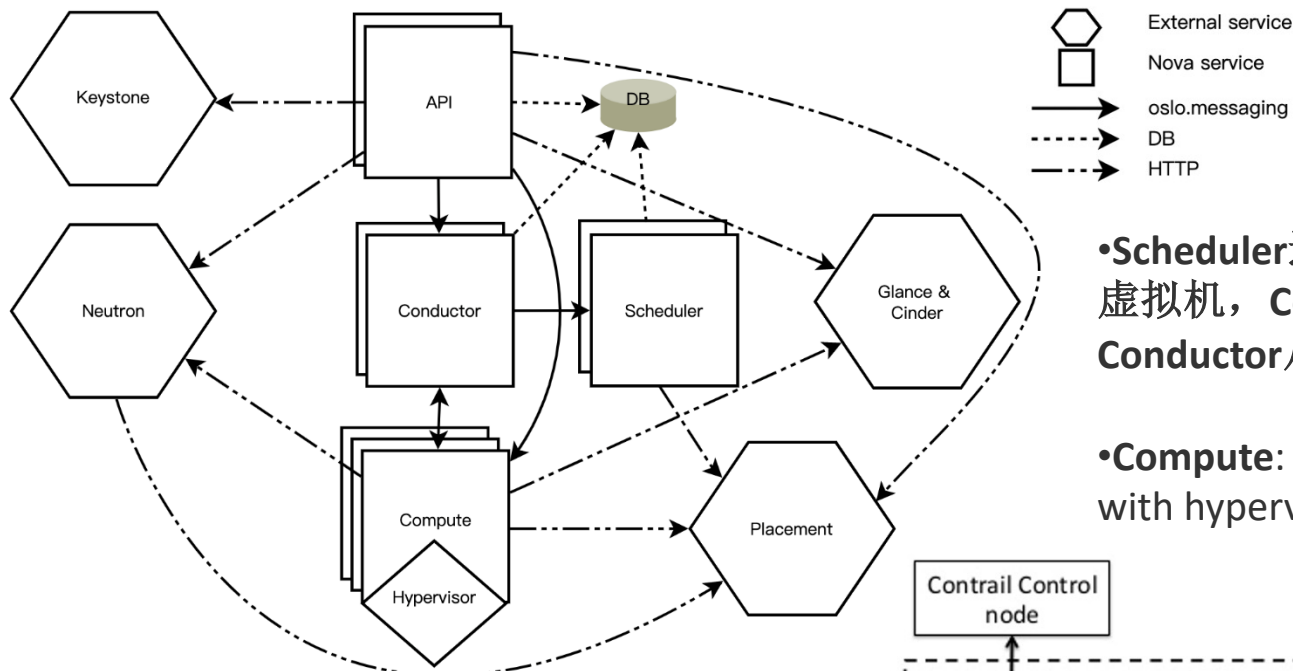
目录

- OpenStack简介
 - 第一印象
 - 加深印象
 - 从概念模型到物理部署
 - 案例：虚拟机创建过程
- 云控制器：Keystone、Nova及其他服务
- 计算节点：Nova、Magnum
- 存储节点：Swift, Cinder
- 网络节点：Neutron
- 重审案例
- 部署
 - DevOps
 - OpenStack和DevOps



计算节点——Nova-compute总览

- 计算节点运行nova-compute服务，负责启动和终止虚拟机，通过消息总线监听虚拟机相关请求

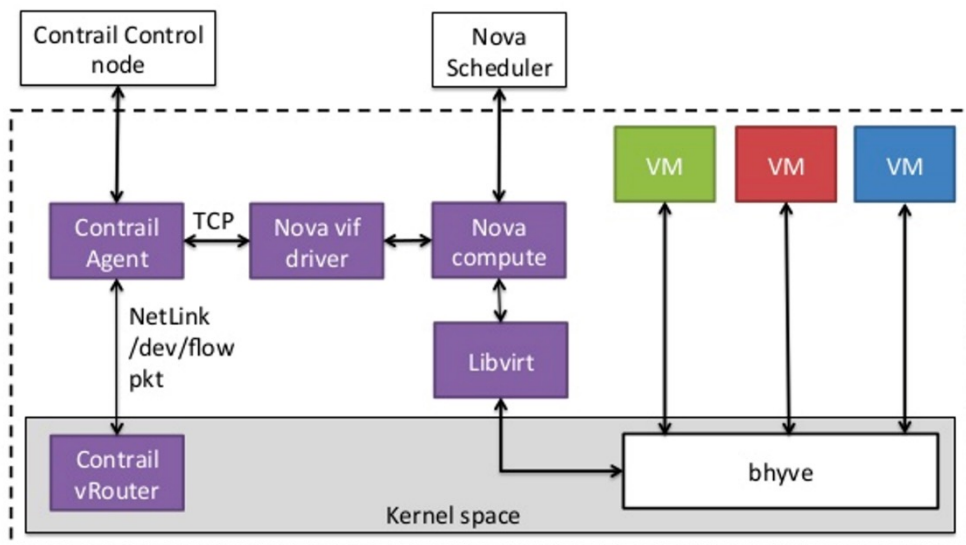


• **Scheduler**通过MQ通知**Compute**创建虚拟机，**Compute**通过MQ请求**Conductor**从DB获取虚拟机相关信息

• **Compute**: Manages communication with hypervisor and virtual machines.

<https://docs.openstack.org/nova/latest/admin/architecture.html>

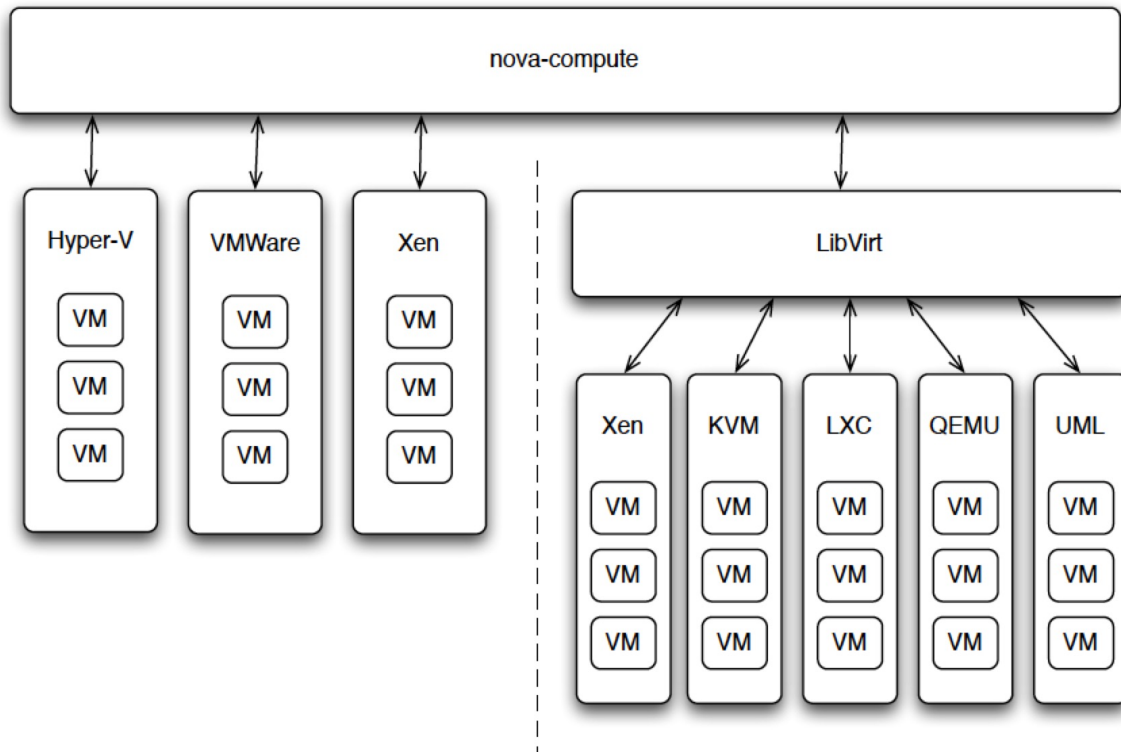
Hypervisor是Nova计算节点的核心，
又称VMM



计算节点——核心：Hypervisor

- Nova-compute通过API server控制节点上的Hypervisor
- 支持多种Hypervisor, 通过配置文件设置选择哪一个hypervisor

`compute_driver = libvirt.LibvirtDriver`



`compute_driver`

Type

string

Default

<None>

Defines which driver to use for controlling virtualization.

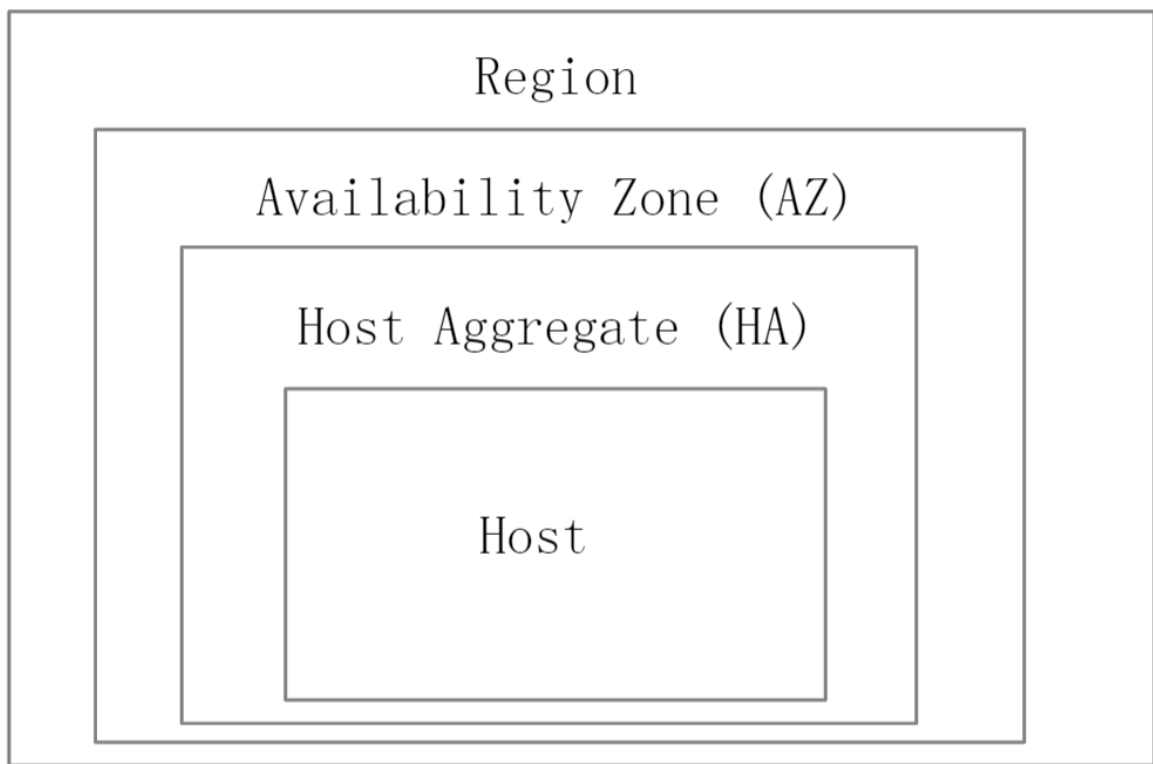
Possible values:

- `libvirt.LibvirtDriver`
- `fake.FakeDriver`
- `ironic.IronicDriver`
- `vmwareapi.VMwareVCDriver`
- `hyperv.HyperVDriver`
- `zvm.ZVMDriver`



计算节点——Nova相关概念（1）

- Nova中对主机有一个层级划分——高于一台物理主机



- Region: 地理区域
- Availability Zone: 对某个地理区域的用户而言的可用区域，可理解为是为了管理方便
- Host Aggregate : 按照组成区域将同一类型物理主机放到一起
- Host: 具体的物理主机，其上可运行多个虚拟机



计算节点——Nova相关概念（2）

名称	简介	说明
Server/Instance	虚拟机	Nova管理提供的云服务资源。Nova中最重要的数据对象。
Server metadata	虚拟机元数据	通常用于为虚拟机附加必要描述信息。key/value。
Flavor	虚拟机规格模板	用于定义一种虚拟机类型，如一种具有2个VCPU、4GB内存、40GB本地存储空间虚拟机。Flavor由系统管理员创建，供普通用户在创建虚拟机时使用。
Quota	资源配额	用于指定租户最多能够使用的逻辑资源上限。
Hypervisor / node	节点	对于KVM、Xen等虚拟化技术，一个node即对应于一个物理主机。对于vCenter，一个node对应于一个cluster。
Host	主机	对于KVM、Xen等虚拟化技术，一个host即对应于一个物理主机，同时对应于一个node。对于vCenter，一个host对应于一套vCenter部署。
Host Aggregate	主机聚合	一个HA内包含若干host。一个HA内的物理主机通常具有相同的CPU型号等物理资源特性。
Server Group	虚拟机亲和性/反亲和组	同一个亲和性组的虚拟机，在创建时会被调度到相同的物理主机上。同一个反亲和性组的虚拟机，在创建时会被调度到不同的物理主机上。
Service	Nova各个服务	管理nova相关服务的状态，包括nova-compute，nova-conductor，nova-scheduler，nova-novncproxy，nova-consoleauth,nova-console



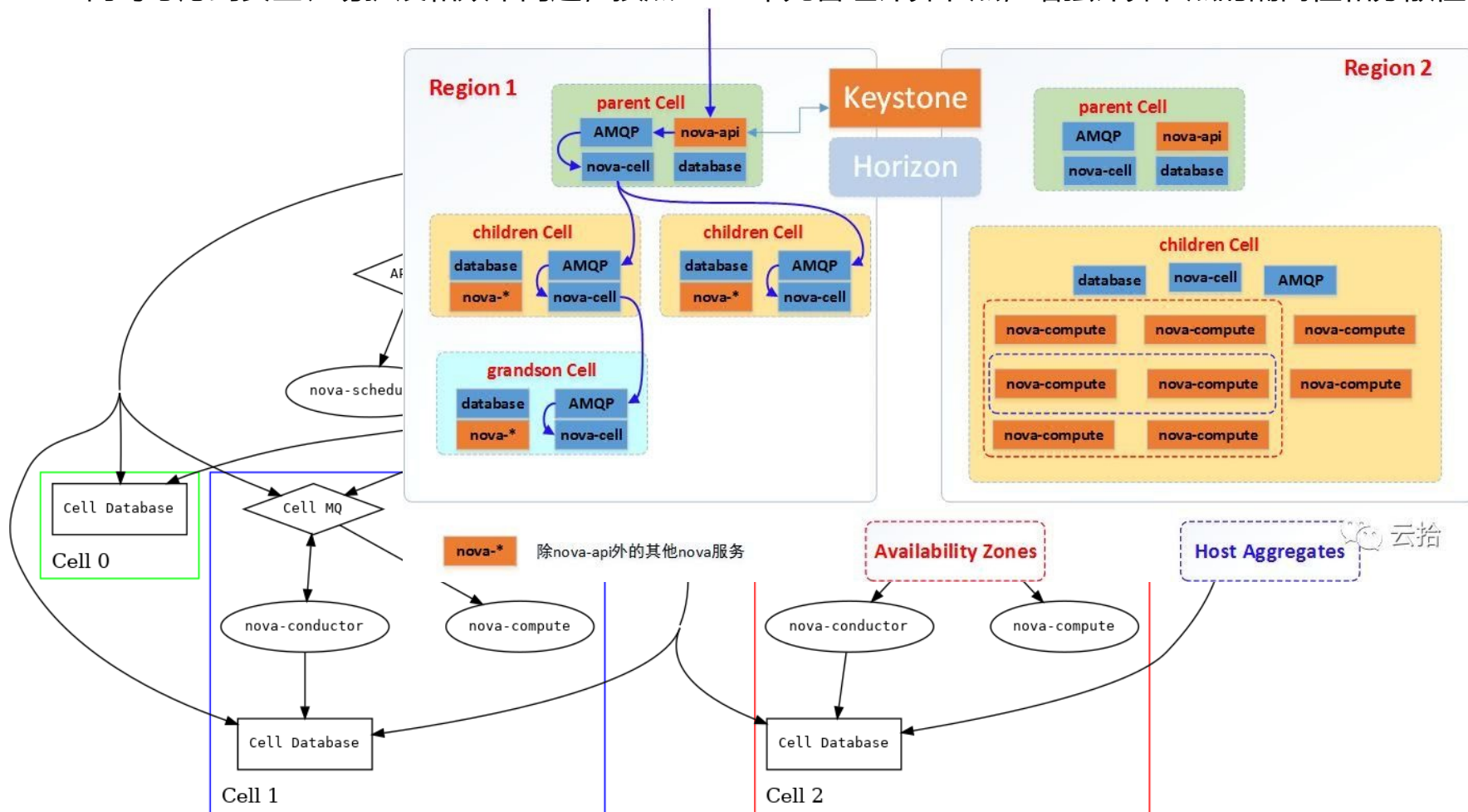
计算节点——Nova操作

分组	说明
虚拟机生命周期管理	虚拟机创建、删除、启动、关机、重启、重建、规格更改、暂停、解除暂停、挂起、继续、迁移、在线迁移、锁定、解锁、疏散、拯救、解拯救、搁置、删除搁置、恢复搁置、备份、虚拟机导出镜像、列表、详细信息、信息查询更改、密码修改
卷和快照管理操作	本质上是对Cinder API的封装。卷创建、删除、列表、详细信息查询。快照创建、删除、列表、详细信息查询
虚拟机卷操作	虚拟机挂卷、虚拟机卸卷、虚拟机挂卷列表、虚拟机挂卷详细信息查询
虚拟网络操作	本质上是对Neutron API的封装。虚拟网络创建、删除、列表、详细信息查询
虚拟机虚拟网卡操作	虚拟机挂载网卡、虚拟机卸载网卡、虚拟机网卡列表。
虚拟机镜像的操作	本质上是对Glance API的封装，支持镜像的创建、删除、列表、详细信息查询。
虚拟机HA	自研虚拟机的可靠性操作。可手动触发(FS5.1)。
其他资源其他操作	Flavor，主机组，keypairs，quota等



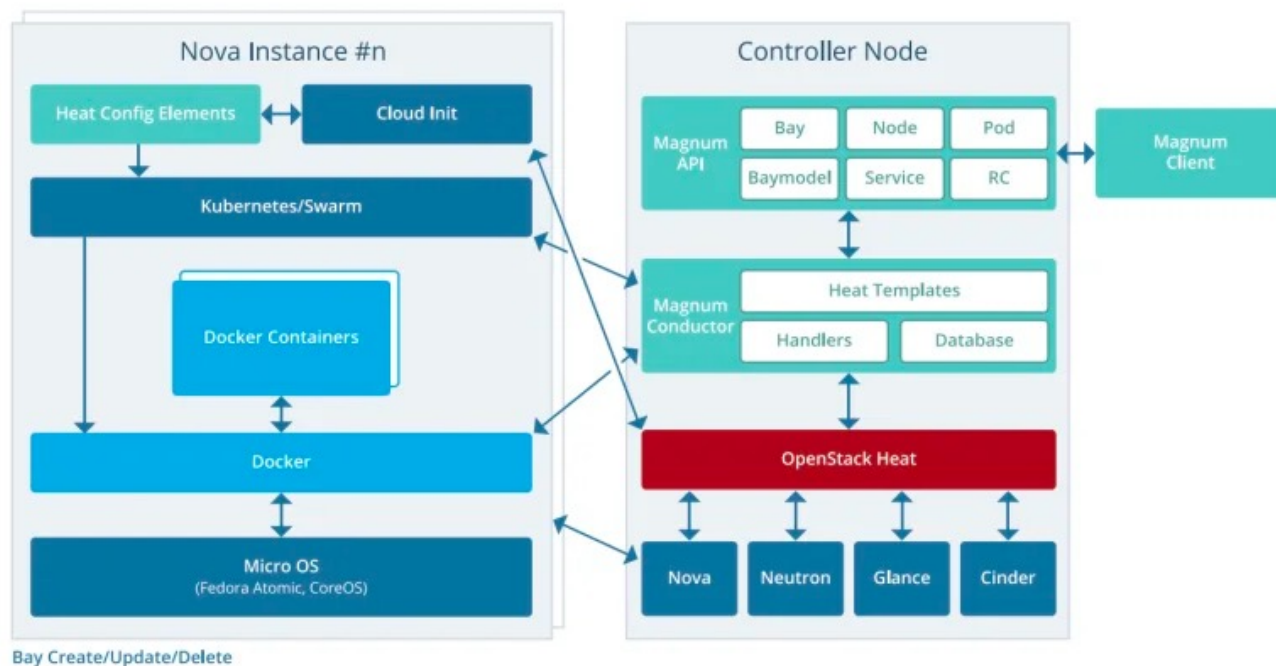
计算节点——Nova单元

- 所有计算节点都要与消息总线 and 数据库服务器通信，随着计算节点增多，使得消息队列和数据库过载
- 同时考虑到安全、易扩展和效率问题，按照Nova单元管理计算节点，增强计算节点的隔离性和分散性



计算节点——不止Nova，还有Magnum

- 现实需求：在计算节点编排容器
- 解决方案1：在Nova中添加可以管理容器生命周期的Hypervisor
- 问题：和虚拟机不同，在容器中托管应用程序往往要部署多个容器，每个容器只运行一个进程，这些容器化的进程相互协作提供完整的应用程序功能。容器需要被成组地创建，需要配置网络连接用于协作进程之间的通信，还需要存储——独立为专门提供容器集群服务的项目：Magnum
- 项目目的：提供容器编排引擎(COE)以方便部署和管理容器集群



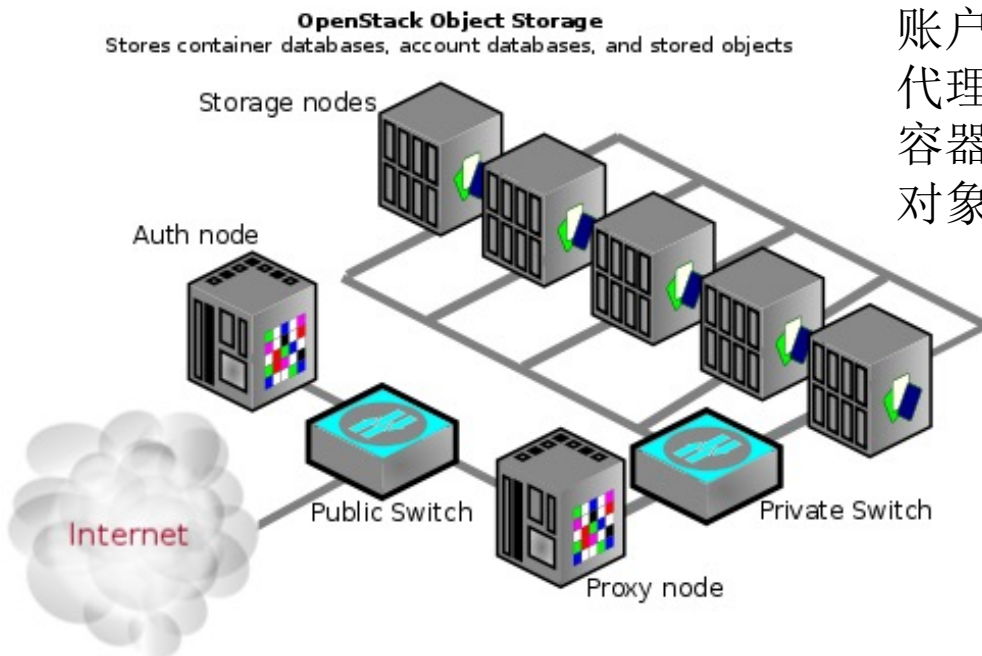
目录

- OpenStack简介
 - 第一印象
 - 加深印象
 - 从概念模型到物理部署
 - 案例：虚拟机创建过程
- 云控制器：Keystone、Nova及其他服务
- 计算节点：Nova、Magnum
- 存储节点：Swift, Cinder
- 网络节点：Neutron
- 重审案例
- 部署
 - DevOps
 - OpenStack和DevOps



存储节点——对象存储Swift

- 数据与其副本作为二进制大对象存储在对象存储服务器上
- 对象存储没有文件存储的层次结构，对象存储在扁平的命名空间——一种特殊的文件系统
- 使用REST或SOAP等API访问对象存储，而不是文件协议
- 不适用于高性能要求以及经常更改的结构化数据，例如数据库

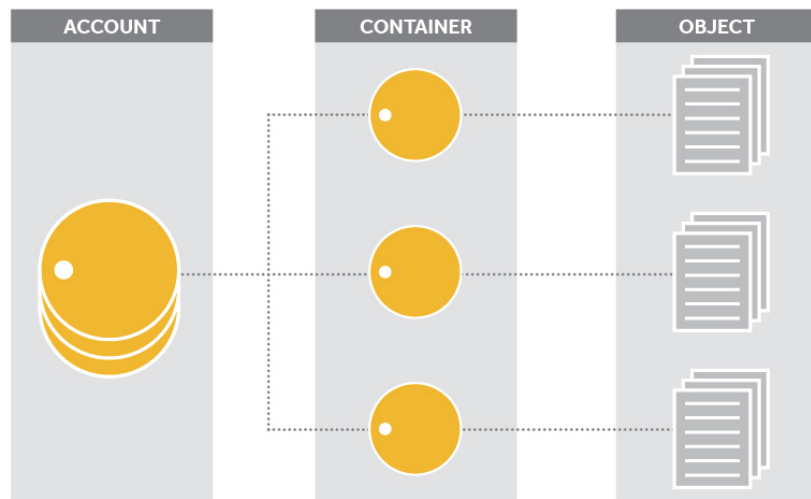


账户服务器：租户、身份认证

代理服务器：运行API

容器服务器：“桶”，一人多桶，一桶多对象

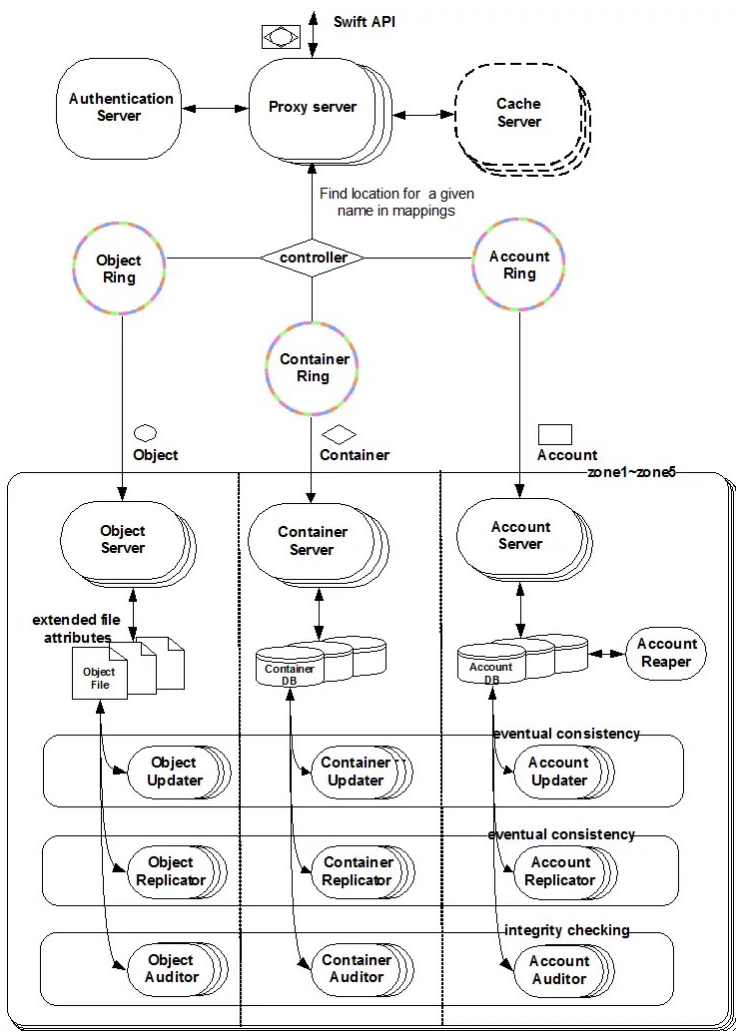
对象服务器：一个对象只能在一个桶中



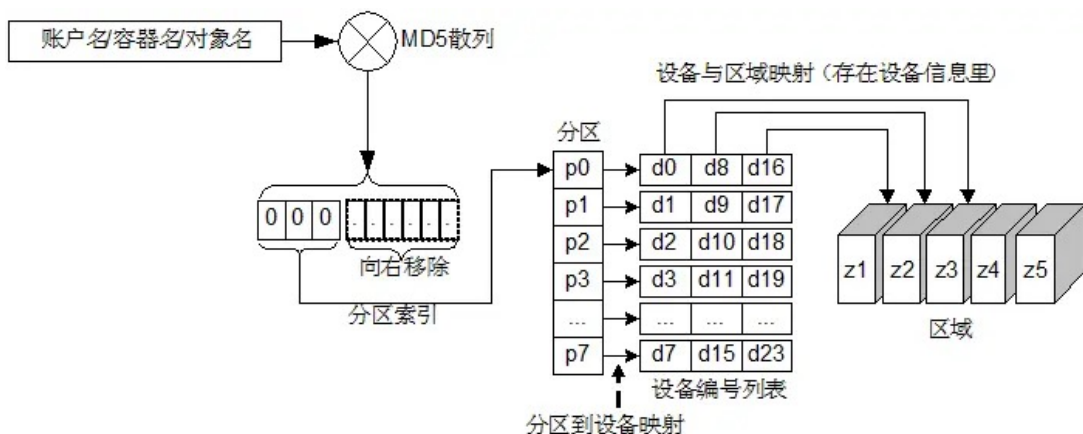
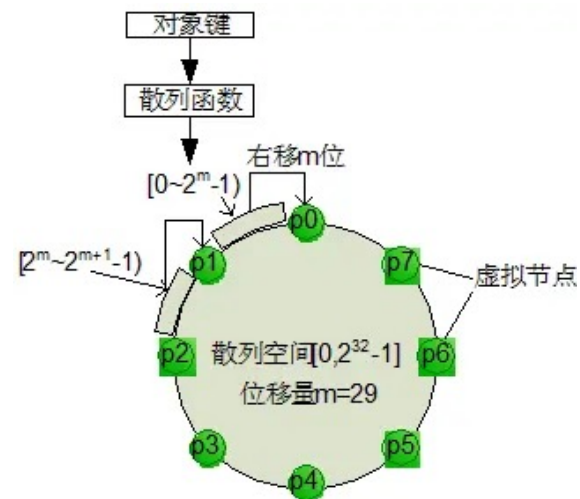
Swift使用环（ring）将账户、容器和对象映射到集群上的物理位置。账户环、容器环、对象环。

存储节点——对象存储Swift——一致性Hash环

- 使用一致性Hash环建立数据和虚拟节点、物理节点的映射



通过hash将对象映射到环上，可以找到对象对应的虚拟节点（partition）。然后通过独特的数据结构Ring（环）再将虚拟节点映射到实际的物理存储设备上，完成寻址过程，实现了对象到dev的计算寻址

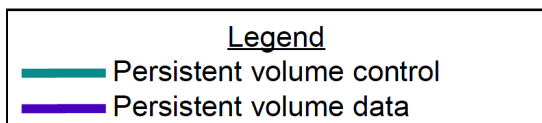
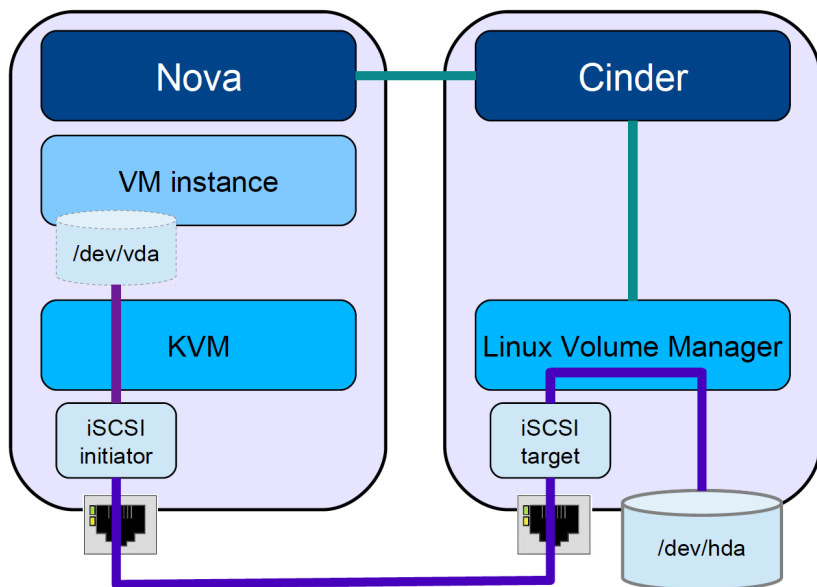


Swift详解 <https://www.jianshu.com/p/7e60c96232f4>

存储节点——块存储Cinder

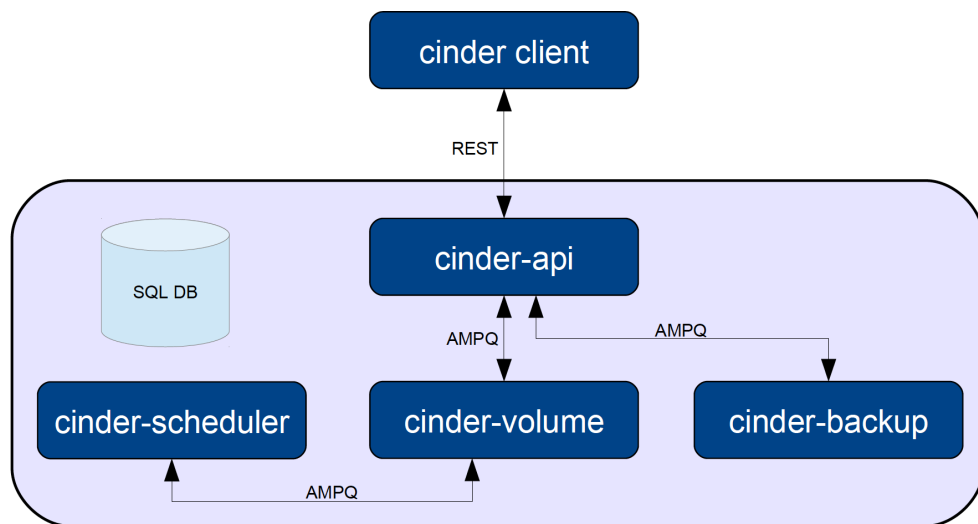
- 为云平台提供统一接口，按需分配的，持久化的块存储服务
- 核心功能是对卷的管理，允许对卷、卷的类型、卷的快照、卷备份进行操作
- 为后端不同的存储设备提供了统一的接口，不同的块设备服务厂商在Cinder 中实现其驱动支持以与OpenStack进行整合

计算节点和Cinder的网络连接

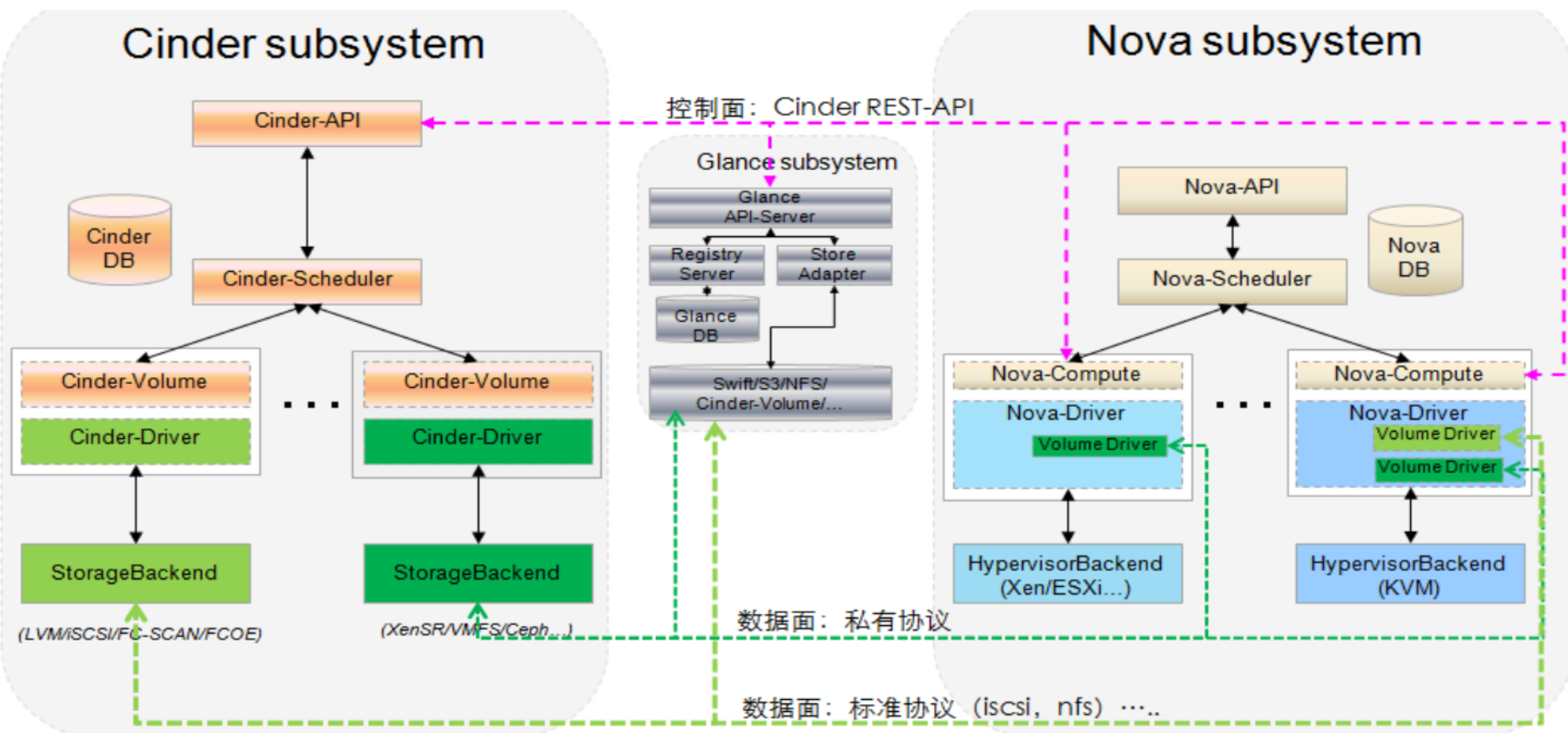


块存储作为虚拟机硬盘使用时，必须先分区、创建文件系统，再挂载到虚拟机文件系统层次结构中。

Cinder内部架构



存储节点——块存储Cinder



目录

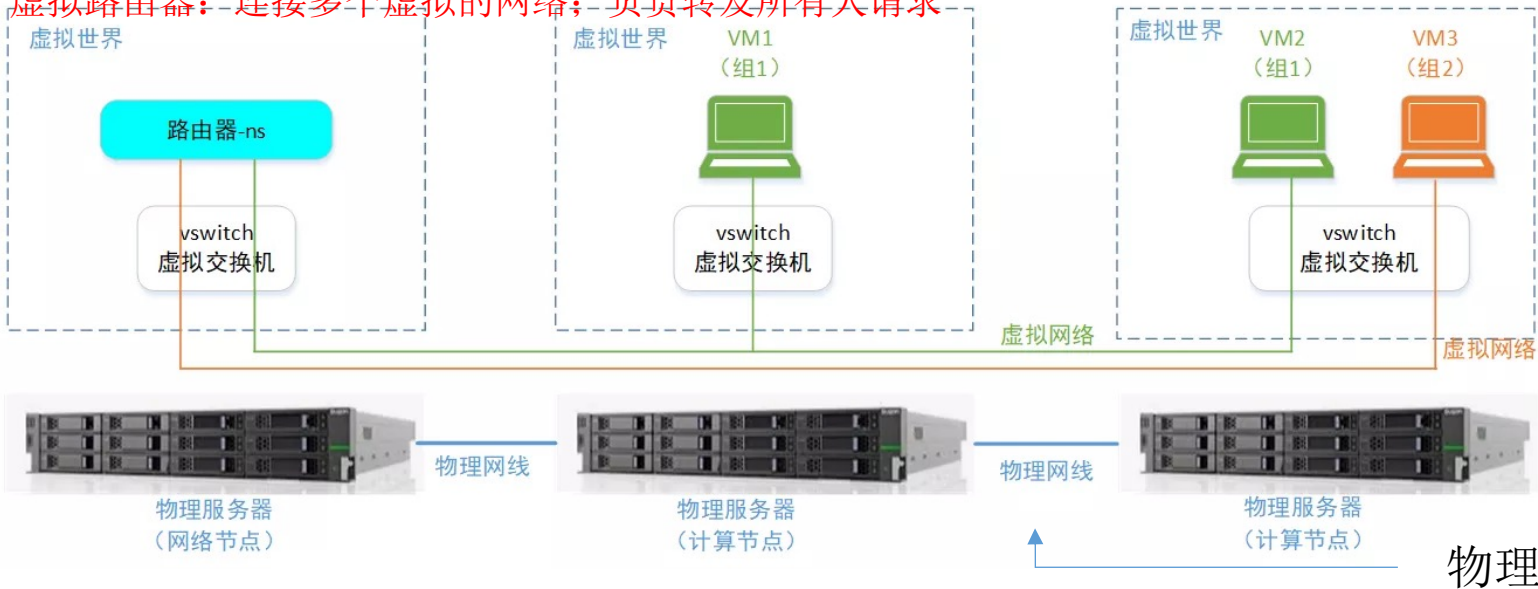
- OpenStack简介
 - 第一印象
 - 加深印象
 - 从概念模型到物理部署
 - 案例：虚拟机创建过程
- 云控制器：Keystone、Nova及其他服务
- 计算节点：Nova、Magnum
- 存储节点：Swift, Cinder
- 网络节点：Neutron
- 重审案例
- 部署
 - DevOps
 - OpenStack和DevOps



网络连接 (1)

路由器：连接多个独立的网络

虚拟路由器：连接多个虚拟的网络；负责转发所有人请求



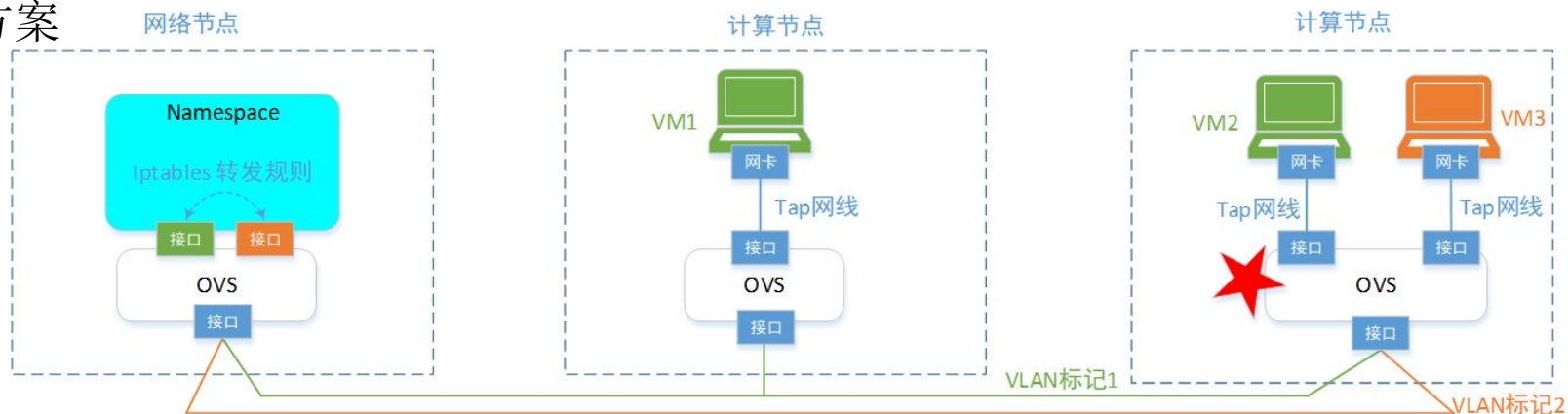
1. 每一个租户有自己的虚拟网络

2. 不同租户间的虚拟网络要互通

↑ 矛盾

物理线路只有一条

解决方案



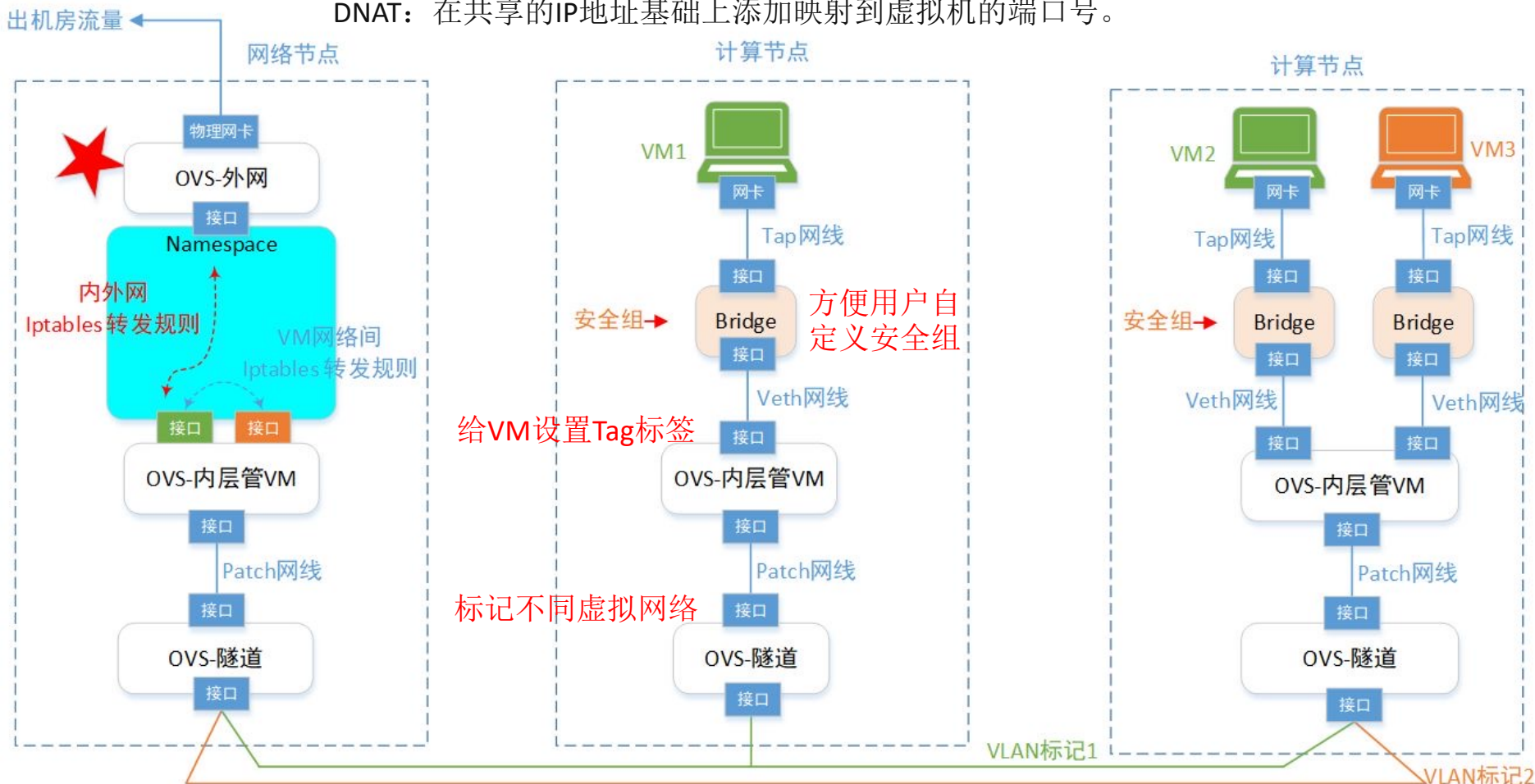
给每个物理服务器上增加一个OVS虚拟交换机；
然后每个VM都连到OVS端口上，每个端口则按照分组，打上对应的VLAN标签。

网络连接 (2)

浮动IP: 每一台虚拟机都动态分配一个对外的IP地址; 外部通过IP访问到虚拟机, 虚拟机通过该IP访问外部网络。

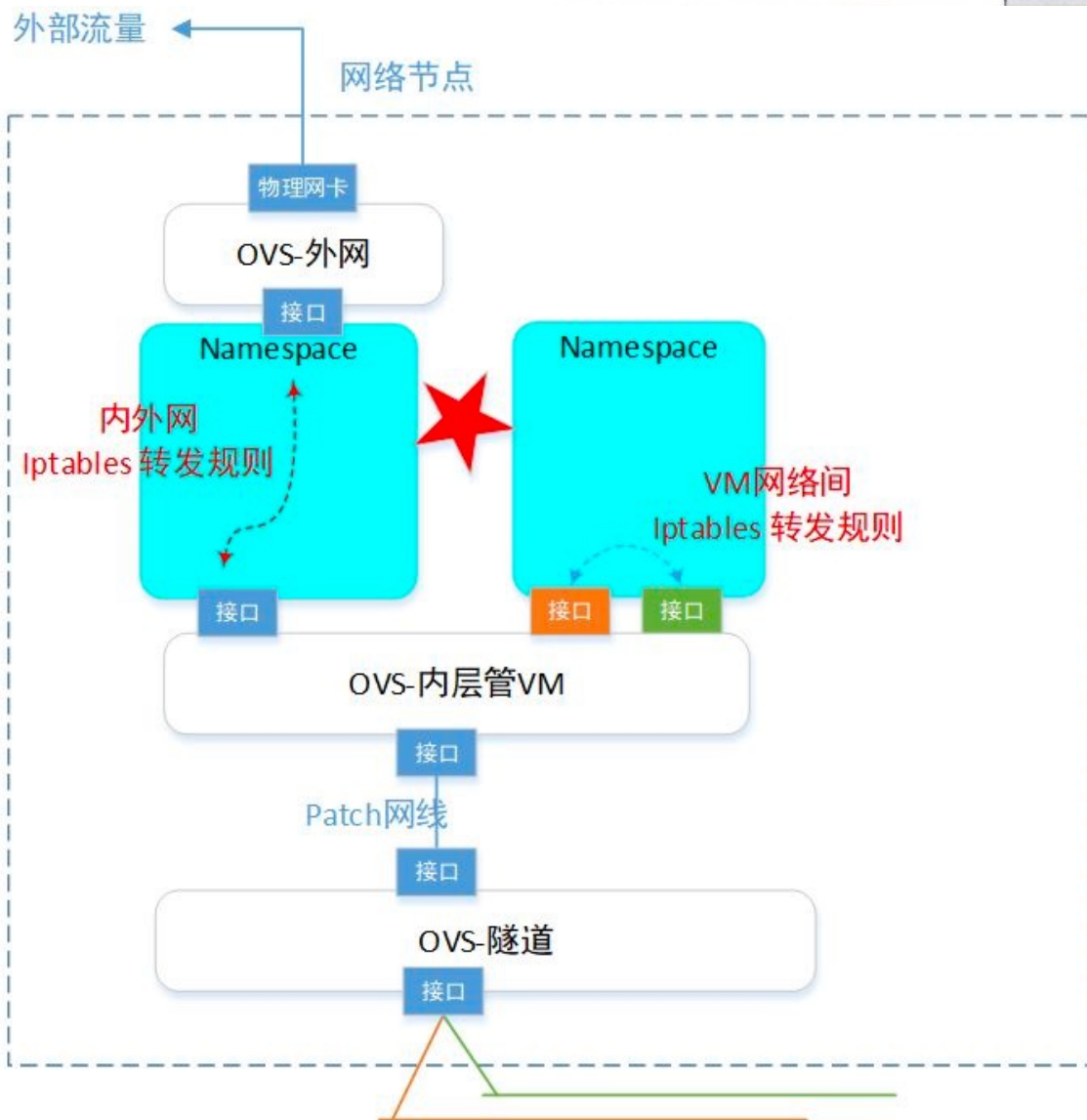
SNAT: 虚拟机共享对外的IP地址, 实现访问外部网络, 但是外部无法通过IP地址定位到具体虚拟机。

DNAT: 在共享的IP地址基础上添加映射到虚拟机的端口号。

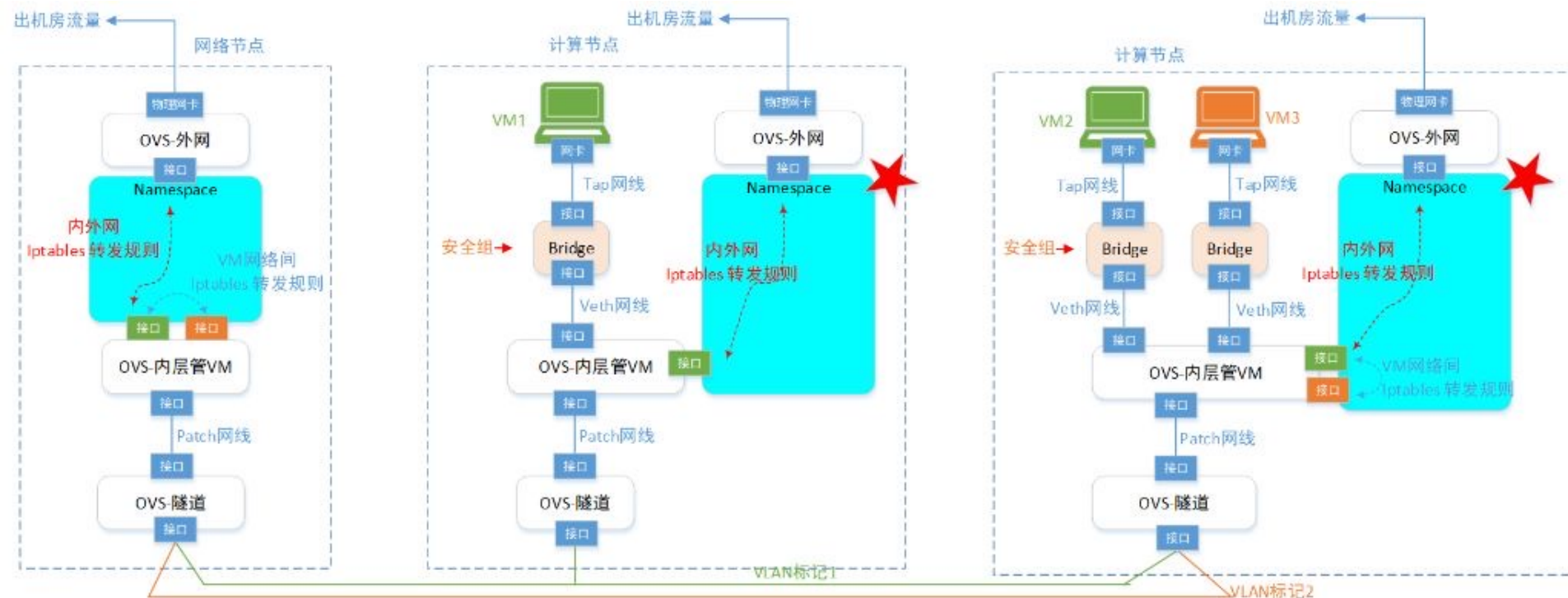


Neutron组件

- API服务器：运行在云控制器上
 - 基于插件的架构
 - API服务将请求转发到特定插件，并
 - 提供了基于开源技术的插件和代理
 - 网络节点提供资源实现网络服务：
- 核心插件：创建和管理网络、端口
- 服务插件：实现高阶网络服务，路由
- 代理：计算和网络节点上，通过消息队列实现
 - L2代理：在计算和网络节点上，将虚拟机网络接口与物理网络接口绑定
 - DHCP代理、L3代理和VPN代理
- 网络节点L3代理：配置虚拟路由器
 - 虚拟网络之间转发
 - 外部网络和虚拟网络之间转发



Distributed Virtual Routing (DVR)

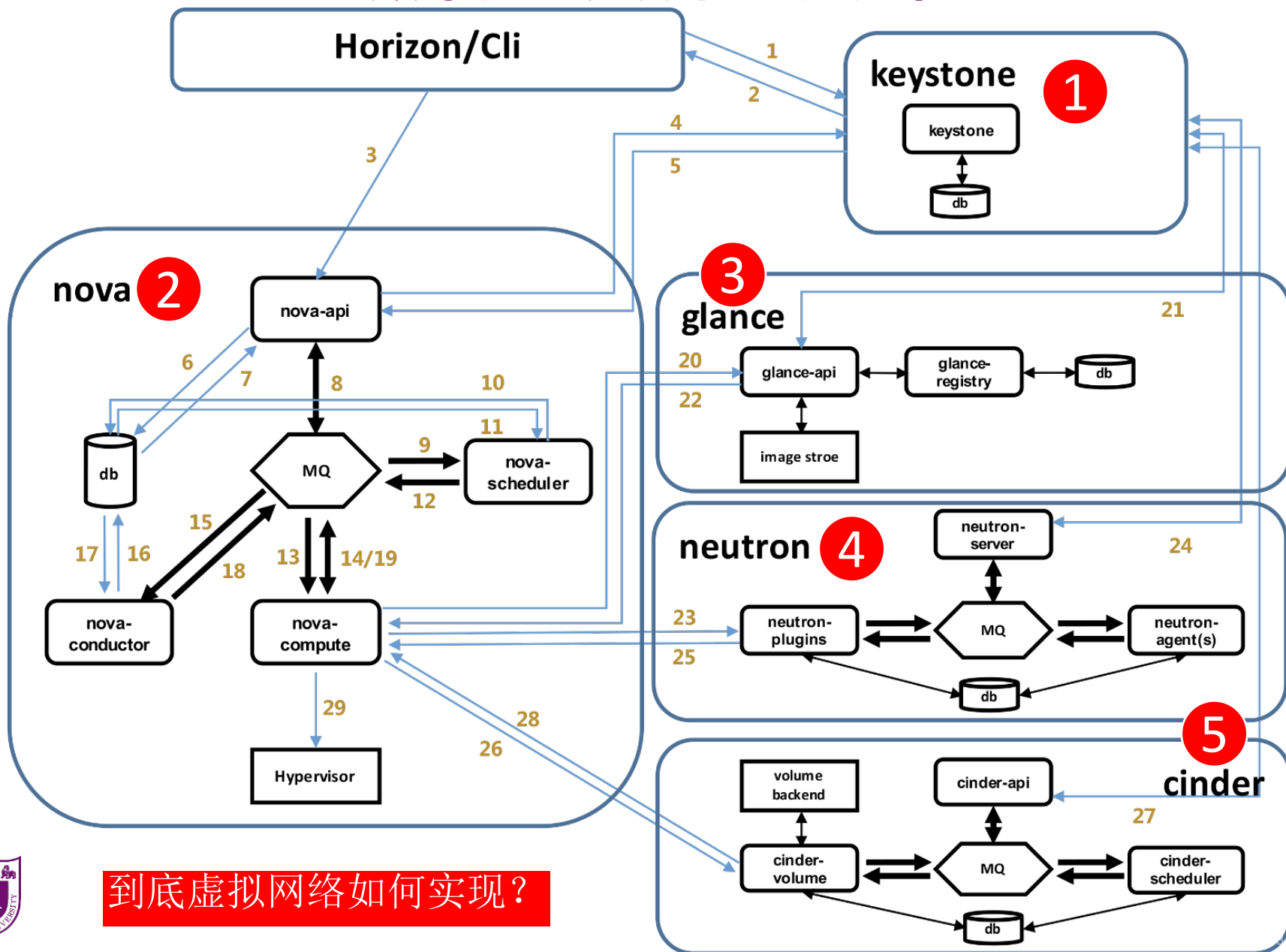


为了降低网络节点的负载，同时提高可扩展性，OpenStack在Juno版本引入了DVR特性，DVR部署在计算节点上。

计算节点上的VM使用floatingIP访问Internet，不必经过网络节点，直接从计算节点的DVR就可以访问。



回顾案例：虚拟机创建过程



到底虚拟网络如何实现？



目录

- OpenStack简介
 - 第一印象
 - 加深印象
 - 从概念模型到物理部署
 - 案例：虚拟机创建过程
- 云控制器：Keystone、Nova及其他服务
- 计算节点：Nova、Magnum
- 存储节点：Swift, Cinder
- 网络节点：Neutron
- 重审案例
- 部署
 - DevOps
 - OpenStack和DevOps



部署——DevOps

- 矛盾
 - 对于运维来说，稳定压倒一切，新 Feature 越少越好。而对于研发来说，却希望能开发更多的功能。
- DevOps 的理念
 - 让研发（Development）和运维（Operations）一体化，让团队从业务需求出发，向着同一个目标前进。
- DevOps 是一种软件研发管理的思想，方法论，追求的是一种没有隔阂的理想的研究协作的状态，可能涉及到的角色有开发、测试、产品、项目管理、运维等等。

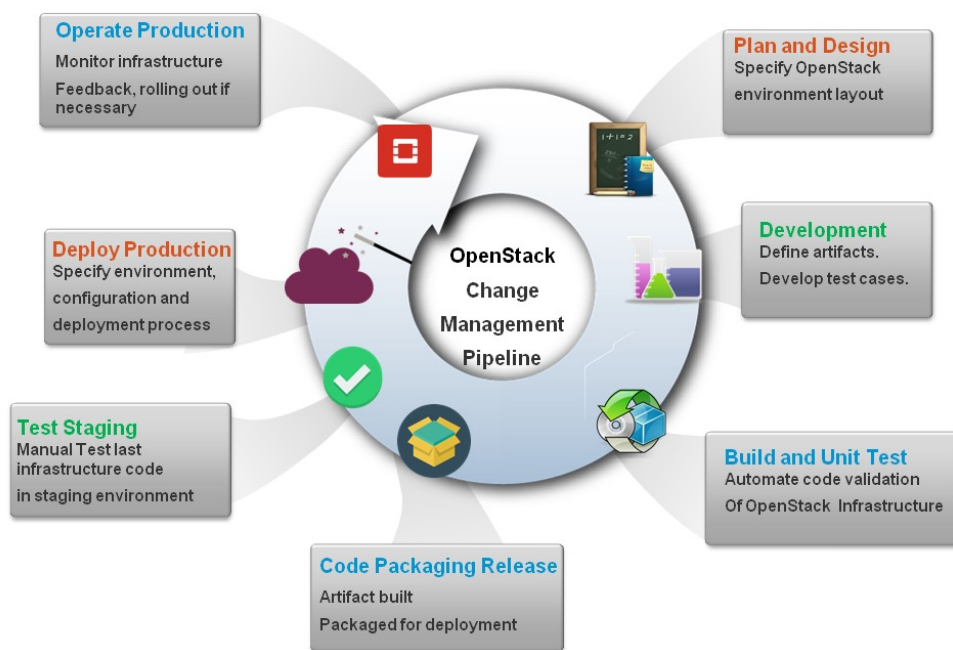


DevOps（Development和Operations的组合）是一组过程、方法与系统的统称，用于促进开发（应用程序/软件工程）、技术运营和质量保障（QA）部门之间的沟通、协作与整合。

部署——DevOps和Openstack

- 要求开发人员、网络工程师和运维人员必须相互协作，来部署、运维和维护OpenStack云基础架构
- OpenStack和DevOps结合
 - 要求
 - OpenStack软件和部署云基础架构的持续增长的复杂性必须被简化
 - 方式
 - 基础架构中的所有内容都必须自动化
 - 将OpenStack分解为多个部分
 - 要点
 - 简化和模块化OpenStack服务；像开发构建模块一样开发OpenStack服务
 - 在不影响整个系统的情况下，促进服务的定制和改进
 - 使用正确的工具来构建服务，确保服务在相同输入情况下输出相同结果
 - 将服务愿景从如何做切换到要做什么

基础架构代码生命周期管理



部署——Ansible

- 基础架构自动化引擎：安装Ansible自身部署系统和管理的目标系统

- 模块

- 封装系统资源或操作的组件，对资源及其属性进行建模；通过执行模块改变目标系统状态；执行后清除

- 变量

- 代表更新中的动态部分，可用于修改模块行为；可以用户自定义，作为模块一部分

- 清单

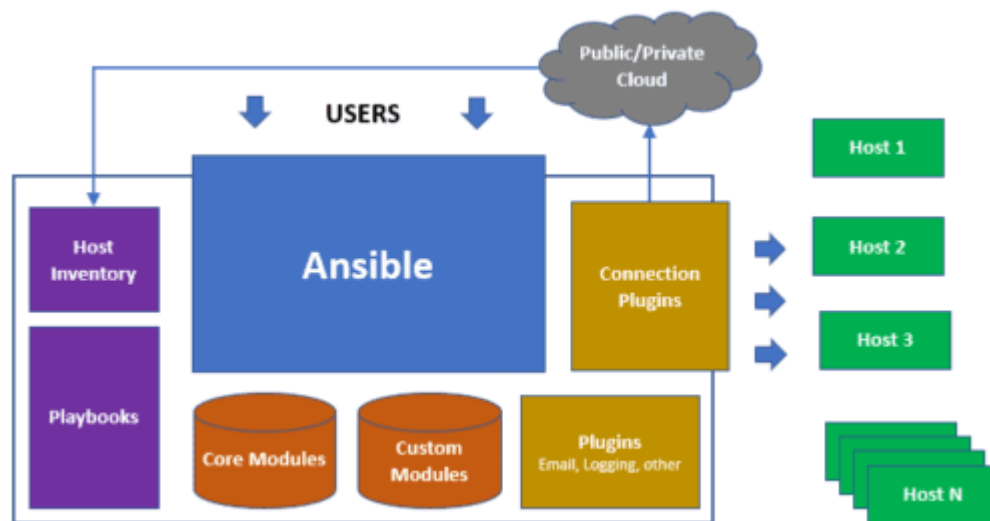
- 由Ansible管理的主机列表；支持将主机分类到多个组中；同一个主机可以出现在多个分段中

- 角色

- 表示在一组主机上配置服务时必须执行的任务集合；封装了在主机上部署服务所需的任务、变量、处理程序和其他相关功能；在Web服务器集群中为主机分配角色：Web服务器，数据库服务器，负载均衡器等

- Playbook

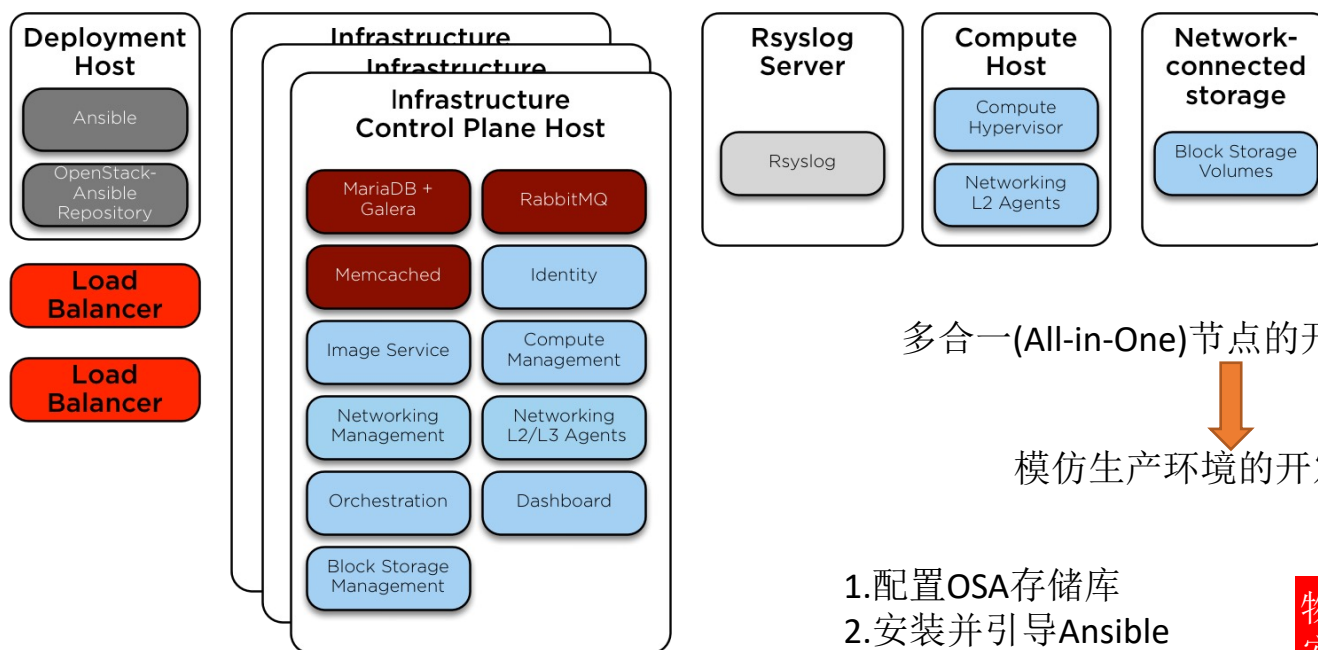
- Ansible的主要配置文件；描述了完整的系统部署计划，由一系列任务组成；用YAML编写
 - Yet Another Markup Language



部署——OpenStack Ansible

- 自动化部署OpenStack
- 聚焦于提供多个角色和playbook，用于部署可扩展的、可理解投入生产的OpenStack环境

Host and Service Layout - Production Environment



多合一(All-in-One)节点的开发者模式安装



模仿生产环境的开发测试环境

- 1.配置OSA存储库
- 2.安装并引导Ansible
- 3.初始化主机引导程序
- 4.运行Playbook

物理环境构建、开发机初始化、安装Git、执行引导程序仍需要手动完成

总结

- OpenStack——云平台构建、管理和监控系统

- 体系架构

- 云控制器
- 计算节点
- 存储节点
- 网络节点

OpenStack组件和体系架构的关系？

- 如何部署——DevOps理念

- 一些关键问题

- 如何新建虚拟机？组件之间的关联是怎样的？
- 虚拟机如何访问外网？
- 外网如何访问虚拟机？
- 虚拟机如何访问虚拟机？

其他内容：

OpenStack的高可用和容错机制
OpenStack集群监控和故障排查
OpenStack ELK日志处理系统
OpenStack基准测试和性能调优



谢 谢



南京大學
NANJING UNIVERSITY